
**Estimación automática de Parámetros
de Síntesis FM
para la obtención de Timbres Musicales
mediante Redes Neuronales**

Automatic Estimation of FM Synthesis
Parameters for Sound Matching using Neural
Networks



**Trabajo de Fin de Grado
Curso 2025–2026**

Autores

David Cendejas Rodríguez
Ángel Jiménez Izquierdo

Directores

Miguel Gómez-Zamalloa Gil
Jaime Sánchez Hernández

**Grado en Ingeniería de Computadores
Grado en Ingeniería Informática
Facultad de Informática
Universidad Complutense de Madrid**

**Estimación automática de Parámetros de Síntesis
FM
para la obtención de Timbres Musicales mediante
Redes Neuronales**

Automatic Estimation of FM Synthesis Parameters for
Sound Matching using Neural Networks

**Trabajo de Fin de Grado en
Ingeniería de Computadores / Ingeniería Informática**

Autores

David Cendejas Rodríguez
Ángel Jiménez Izquierdo

Directores

Miguel Gómez-Zamalloa Gil
Jaime Sánchez Hernández

Convocatoria: *Junio 2026*

**Grado en Ingeniería de Computadores
Grado en Ingeniería Informática
Facultad de Informática
Universidad Complutense de Madrid**

25 de Mayo de 2026

Dedicatoria

*A Miguel y a Jaime, por su paciencia y por
habernos introducido en el mundo de la
informática musical y la investigación
académica.*

*A Carlos, por su guía en las tareas de machine
learning.*

*A nuestras familias y amigos, por su cariño y
apoyo incondicional.*

Agradecimientos

A cualquier persona involucrada en el despegue de nuestras carreras profesionales como informáticos a lo largo de estos cuatro años: profesores, compañeros, familia y amigos.

Resumen

Estimación automática de Parámetros de Síntesis FM para la obtención de Timbres Musicales mediante Redes Neuronales

La invención de los **sintetizadores** en los años 60 revolucionó el mundo de la música, antes dominada por los instrumentos analógicos tradicionales. La llegada de la **música sintética** y del ordenador como elemento central de la producción moderna han definido la forma de trabajar de los artistas modificando su sonido y procesos creativos. Desde entonces los productores musicales y diseñadores de sonido han pasado años aprendiendo a dominar los sintetizadores para definir y crear timbres musicales nunca antes vistos, sin embargo, dicho proceso a menudo requiere de mucha maestría, pues los distintos tipos de síntesis suelen ser complejos, modulares y escalables, lo que lo convierte en una tarea altamente complicada para la mayoría de los músicos. Es común que incluso los mejores diseñadores de sonido encuentren dificultad en **replicar otros sonidos sintéticos**, pues dependen directamente del hardware utilizado y de la secuencia de efectos y modulaciones sonoras aplicadas.

Es por ello que la aplicación del **aprendizaje profundo** a la síntesis abre todo un abanico de posibilidades al delegar la detección de patrones y conocimiento técnico requerido a la **inteligencia artificial**, permitiendo al artista ocuparse de la experimentación y la creación que le pertenece. Mediante el uso de **Redes Neuronales Convolucionales (CNNs)**, este proyecto busca automatizar la estimación de parámetros en la compleja **síntesis FM**. Así, la IA resuelve la barrera técnica de replicar un sonido (**sound matching**), convirtiéndose en una herramienta directa al servicio de la creatividad.

Todo el código desarrollado se encuentra en nuestro repositorio de GitHub:
<https://github.com/Angelji06/TFG-SoundMatchingFM>

Palabras clave

Replicación Sonora, Síntesis FM, Aprendizaje Profundo, Redes Neuronales Convolucionales, Procesamiento de Señales de Audio, Espectrogramas, Python, PyTorch.

Abstract

Automatic Estimation of FM Synthesis Parameters for Sound Matching using Neural Networks

The invention of **synthesizers** in the 1960s revolutionized the music world, previously dominated by traditional analog instruments. The arrival of **synthetic music** and the computer as a central element of modern production have defined the way artists work, modifying their sound and creative processes. Since then, music producers and sound designers have spent years learning to master synthesizers to define and create new musical timbres; however, this process often requires great mastery, as the different types of synthesis are usually complex and scalable, making it a highly complicated task for most musicians. It is common for even the best sound designers to find it difficult to **replicate other synthetic sounds**, as they depend directly on the hardware used and the sequence of effects and sound modulations applied.

That is why the application of **deep learning** to synthesis opens up a whole range of possibilities by delegating pattern detection and the technical knowledge required to **artificial intelligence**, allowing the artist to focus on the experimentation and creation that belongs to them. Through the use of **Convolutional Neural Networks (CNNs)**, this project seeks to automate parameter estimation in complex **FM synthesis**. Thus, AI solves the technical barrier of replicating a sound (**sound matching**), becoming a direct tool at the service of creativity.

All the developed code can be found in our GitHub repository:
<https://github.com/Angelji06/TFG-SoundMatchingFM>

Keywords

Sound Matching, FM Synthesis, Deep Learning, Convolutional Neural Networks, Audio Signal Processing, Spectrograms, Python, PyTorch.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Plan de trabajo	3
2. Estado de la Cuestión	5
2.1. Estado de la cuestión: Audio	5
2.1.1. Síntesis sonora	5
2.1.2. Síntesis por Modulación de Frecuencia (FM)	5
2.1.3. Representación digital del sonido: La onda como vector	7
2.1.4. Representación espectral: STFT y Espectrogramas	7
2.1.5. Escala de Mel	8
2.1.6. <i>Mel Frequency Cepstral Coefficients</i> (MFCC)	8
2.2. Estado de la cuestión: IA aplicada al audio	9
2.2.1. CNNs en el dominio del audio	9
2.2.2. Transformers en el dominio del audio (AST)	9
2.2.3. Inferencia automática de parámetros en síntesis	10
2.2.4. Representación del audio usada en otros proyectos	10
2.2.5. Datasets	11
3. Metodología y Diseño del Sistema	13
3.1. Generación del Dataset	13
3.2. Preprocesamiento de Datos: normalizaciones y espectrogramas	13
3.3. Diseño de la Arquitectura CNN	14
3.4. Funciones de Pérdida	15
3.4.1. El problema de la inyectividad	15
3.4.2. Funciones	16
3.4.3. Modelo de Pruebas simplificado	17
4. Implementación	19
4.1. Herramientas y Librerías: <i>stack</i> tecnológico	19
4.2. <i>Pipeline</i> de Entrenamiento	19

4.3.	Inferencia de parámetros	20
4.4.	Interfaz Gráfica de la aplicación	21
4.5.	Estructura del Código	21
5.	Evolución Arquitectónica	25
5.1.	Exploración de Paradigmas: Clasificación vs. Regresión	25
5.1.1.	Enfoque de Clasificación	25
5.1.2.	Enfoque de Regresión	26
5.2.	El Problema de la Inyectividad y la Necesidad de una Arquitectura Compleja	27
5.2.1.	Prototipo 4: Red Convolutiva de Doble Cabeza	28
5.2.2.	Prototipo 4: Función de Pérdida Híbrida	29
6.	Experimentación Previa	31
6.1.	Limitaciones de Fréchet Audio Distance (FAD)	31
6.2.	Validación por Fuerza Bruta (Grid Search)	31
6.2.1.	Diseño experimental y parametrización del Grid Search	32
6.2.2.	Espectrograma de Mel	32
6.2.3.	Espectrograma MFCC	34
6.2.4.	Conclusiones	35
7.	Arquitectura Final	37
7.1.	Expansión Paramétrica: Evolventes y Generación Aleatoria del Dataset	37
7.1.1.	Generación y Tratamiento del Dataset	37
7.1.2.	Entrenamiento	39
7.1.3.	Inferencia	41
7.1.4.	Evaluación	41
7.2.	Interfaz Gráfica de la Aplicación	42
7.3.	Estructura del Código	43
8.	Resultados y Evaluación	47
8.1.	Definición de los Pipelines Experimentales	48
8.2.	Características del <i>Hardware</i> empleado en el entrenamiento de los modelos	48
8.3.	Evaluación de las Diferentes Arquitecturas y Funciones de Pérdida	48
8.3.1.	P1: CNNRegressorSimple: SmoothL1, STFT lineal	50
8.3.2.	P2: CNNRegressorSimple: SmoothL1, Mel (log-perceptual)	50
8.3.3.	P3: CNNRegressor5: HybridLoss, STFT lineal	53
8.3.4.	P4: CNNRegressor5: HybridLoss, Mel (log-perceptual)	53
8.3.5.	P5: CNNRegressor5: MultiScaleSpectralLoss, STFT lineal	54
8.3.6.	P6: CNNRegressor5: MultiScaleSpectralLoss, Mel (log-perceptual)	54
8.4.	Comparación de los Resultados Obtenidos	54
9.	Conclusiones y Trabajo Futuro	57

10.Introduction	59
10.1. Motivation	59
10.2. Objectives	60
10.3. Work Plan	61
Conclusions and Future Work	63
Contribuciones Personales	65
Bibliografía	67

Índice de figuras

2.1. Yamaha DX7. Fuente (Hispasonic).	6
2.2. Representación del muestreo de una onda. Fuente: Wikipedia (a).	7
2.3. Ejemplo de un espectrograma. Fuente: Wikipedia (b).	8
2.4. Foto del VST Dexed. Fuente: Green Musicians.	11
4.1. Flujo de trabajo del Modelo: desde la síntesis matemática hasta la inferencia y evaluación del modelo.	20
4.2. Primera pantalla de la Interfaz Gráfica de la Aplicación, elección uso modelo o entrenamiento.	23
4.3. Pantalla en la que se pueden hacer pruebas con el modelo seleccionado.	23
4.4. Sintetizador FM de prueba.	24
4.5. Sintetizador FM de prueba.	24
7.1. Flujo de trabajo del Prototipo 5: desde la síntesis matemática hasta la inferencia y evaluación del modelo.	38
7.2. Primera pantalla de la Interfaz Gráfica de la Aplicación, elección uso modelo o entrenamiento.	44
7.3. Pantalla en la que se pueden hacer pruebas con el modelo seleccionado.	44
7.4. Sintetizador FM de prueba.	45
7.5. Sintetizador FM de prueba.	45
8.1. Representación de las diferentes combinaciones de Arquitecturas y Funciones de Pérdida.	49
8.2. Graficas que muestran los resultados de evaluar el conjunto de tests sobre el modelo P2 con las métricas Mel L1 y MCD.	51
8.3. Gráfico obtenido en la evaluación de P2 mediante <i>benchmark</i>	51
8.4. Espectrograma resultado de la predicción del audio de Benchmark: <i>muy_bajo</i>	52
8.5. Espectrograma resultado de la predicción del audio de Benchmark: <i>muy_bajo</i>	52
8.6. Gráfico obtenido en la evaluación de P3 mediante <i>benchmark</i>	54

Índice de tablas

8.1. Comparativa de métricas paramétricas y acústicas sobre el Test Set para los 6 pipelines. Las flechas ($\downarrow$) indican que un valor menor representa un mejor rendimiento. El Decoder L1 no aplica (N/A) en las arquitecturas simples.	55
--	----

Introducción

“Sueño con instrumentos que obedezcan a mi pensamiento y que, con el aporte de un mundo de sonidos insospechados, se presten a las exigencias de mi ritmo interior.”

— Edgard Varèse

1.1. Motivación

La producción musical moderna ha experimentado una transformación radical en las últimas décadas, consolidando al ordenador y al *software* como los ejes centrales del proceso creativo. En este paradigma digital, la generación de señales de audio artificiales mediante sintetizadores se ha convertido en una herramienta indispensable para artistas, productores y diseñadores de sonido. Históricamente, los primeros sintetizadores analógicos, basados en métodos de síntesis aditiva¹ y sustractiva², ofrecían un flujo de trabajo relativamente accesible; su interfaz física y visual, controlada a través de potenciómetros y filtros, permitía a los músicos esculpir el sonido de una manera intuitiva y directa.

Sin embargo, el panorama de la síntesis cambió drásticamente a finales de los años sesenta con el descubrimiento de la Síntesis por Modulación de Frecuencia (FM) y, de forma masiva, con la comercialización del emblemático **Yamaha DX7** en 1983. Esta tecnología supuso una revolución en el mercado al permitir la creación de timbres metálicos, eléctricos y campanados, acústicamente imposibles de lograr con la tecnología de la época. No obstante, esta innovación trajo consigo un dilema significativo: **su programación es notoriamente contraintuitiva**. A diferencia de los sistemas tradicionales, en la síntesis FM un cambio mínimo en un solo parámetro (como el índice de modulación o la frecuencia de un operador) puede destruir por completo la estructura armónica del sonido. Esta extrema sensibilidad paramétrica y la pronunciada curva de aprendizaje han alejado históricamente a la mayoría de los creadores del diseño sonoro FM, relegándolos al uso de *presets* preconfigurados.

¹La síntesis aditiva busca la construcción de sonidos combinando (sumando, es decir, ‘por adición’) elementos más simples que el sonido original. (Hispasónic, 2026).

²Es un método de síntesis donde una señal es generada por un oscilador y después filtrada (Wikipedia, 2026).

Este nivel de complejidad técnica manifiesta su mayor obstáculo en el proceso conocido como ***Sound Matching*** o igualación tímbrica. Es una situación cotidiana en el estudio de grabación que un productor escuche un sonido específico y desee replicarlo. Realizar esta tarea “a oído” con un sintetizador FM es un proceso de ensayo y error que puede resultar profundamente frustrante. Desde una perspectiva matemática y acústica, este problema radica en la **falta de inyectividad** del motor de síntesis: combinaciones de parámetros drásticamente diferentes pueden generar resultados acústicos que el oído humano percibe como idénticos, mientras que ajustes minúsculos pueden derivar en texturas sonoras completamente opuestas. En consecuencia, el diseño manual de sonidos se convierte en una barrera técnica que interrumpe el flujo creativo del artista.

Ante esta problemática, la Inteligencia Artificial y, en concreto, el **aprendizaje profundo** (*Deep Learning*), se presentan como el puente ideal entre la abstracción creativa del músico y la complejidad matemática del sintetizador. Al delegar la inferencia de parámetros a un modelo computacional, se puede automatizar la búsqueda del sonido objetivo. Para lograrlo, el sistema debe ser capaz de “escuchar” y procesar la señal de una forma asimilable por la red. Mediante la transformación del audio unidimensional a un **espectrograma**, se logra “fotografiar” el sonido, adaptando las frecuencias a una escala logarítmica que imita fielmente la percepción del sistema auditivo humano.

Una vez que el audio se representa como una imagen bidimensional, las **Redes Neuronales Convolucionales (CNNs)** emergen como la arquitectura idónea para abordar el problema. Estas redes están diseñadas para extraer características espaciales y, en el dominio del espectrograma, son excepcionalmente precisas al “ver” y clasificar las densas texturas tímbricas y armónicas del audio.

1.2. Objetivos

Objetivo General:

Desarrollar un sistema basado en **Redes Neuronales Convolucionales (CNNs)** capaz de estimar automáticamente los parámetros de un sintetizador de **modulación de frecuencia (FM)** a partir de una muestra de audio, con el fin de replicar su timbre (*sound matching*). Además, se estudiará y comparará el uso de distintas representaciones del espectrograma, arquitecturas de red y funciones de pérdida.

Objetivos Específicos:

- **Comprensión de la síntesis FM:** Estudiar y comprender la arquitectura paramétrica de la síntesis FM, desde su versión más simple hasta una versión más compleja que involucre envolventes de amplitud y modulación.
- **Revisión bibliográfica:** Investigar y leer la literatura relacionada con el uso de la **Inteligencia Artificial aplicada al audio** para tomar las decisiones de diseño fundamentales del proyecto.
- **Generación del dataset:** Diseñar e implementar un motor de síntesis capaz

de generar un *dataset* masivo de audios sintéticos (*.wav*) y sus correspondientes etiquetas (parámetros), así como su conversión a tensores de PyTorch (*.pt*).

- **Procesamiento de señal:** Establecer un *pipeline* de procesamiento de señales para transformar el audio crudo en representaciones visuales (**espectrogramas**) aptas para la CNN, aplicando las transformaciones y normalizaciones del dato que maximicen su rendimiento.
- **Desarrollo del modelo:** Diseñar, entrenar y optimizar varias arquitecturas de Redes Neuronales Convolucionales y funciones de pérdida en Python usando **PyTorch**.
- **Evaluación psicoacústica:** Evaluar el rendimiento del modelo mediante métricas de error sobre parámetros y medidas de **similitud tímbrica** que simulen la percepción del oído humano.
- **Desarrollo de la interfaz:** Desarrollar un sistema con una interfaz funcional que permita acceder a todas las utilidades del proyecto de manera fácil e intuitiva.

1.3. Plan de trabajo

El desarrollo de este Trabajo de Fin de Grado abarca desde septiembre de 2025 hasta mayo de 2026. La evolución del proyecto a lo largo de estos meses se divide en cinco etapas claramente definidas:

- **Fase 1 (Septiembre - Octubre): Exploración y pruebas de concepto**
Hito principal: Etapa de investigación general sobre Inteligencia Artificial aplicada a la música.
Durante estos meses, comenzamos a asentar las bases técnicas del aprendizaje profundo y su aplicación al audio leyendo la literatura actual. Empezamos haciendo pruebas muy básicas para ver de qué era capaz la IA. Tras barajar opciones como un asistente inteligente de mezcla o un generador de efectos de sonido, finalmente nos orientamos hacia su aplicación con sintetizadores. Los dos primeros prototipos reflejan nuestro progreso en este campo nuevo para nosotros, experimentando con la predicción de formas de onda y la generación de espectrogramas.
- **Fase 2 (Noviembre): Definición del proyecto, *Sound Matching* y primeros prototipos funcionales**
Hito principal: Toma de decisión por el *Sound Matching* FM y logro del primer prototipo real.
Aquí el proyecto tomó su rumbo definitivo. Nos decantamos por la Síntesis por Modulación de Frecuencia (FM) y comenzamos a generar *datasets* masivos etiquetados. Entrenamos un primer modelo de regresión CNN cuyos resultados iniciales fueron malos, lo que nos llevó a reestructurar el código, así como refactorizarlo a Programación Orientada a Objetos (POO) y a crear una Interfaz

Gráfica (GUI) básica. Finalmente, logramos la primera prueba funcional de *sound matching*, aunque todavía con problemas en las frecuencias graves y falta de consistencia.

- **Fase 3 (Diciembre - Febrero): Escalado de complejidad y métricas de evaluación**

Hito principal: Mejora de la síntesis y evaluación del modelo.

Una vez que la base funcionaba, añadimos complejidad incorporando nuevos parametros de envolventes y logaritmos a la generación de sonido. Reimplementamos la generación del dataset para este nuevo paradigma (de 3 a 8 parámetros), además, unificamos el tratamiento del dato en todo el programa para asegurar consistencia en la obtención de los espectrogramas y la normalización necesaria para el correcto funcionamiento de la red neuronal. También implementamos las primeras métricas de validación, la métrica FAD (*Fréchet Audio Distance*) para evaluar los resultados y las herramientas para visualizar y comparar los espectrogramas.

- **Fase 4 (Febrero - Abril): Optimización arquitectónica y comienzo de la memoria**

Hito principal: Cambios profundos en la red neuronal e inicio de la redacción.

Estos fueron los meses en los que más se trabajó en el desarrollo del prototipo final. En esta fase comenzamos a recopilar todo lo anotado y a redactar la memoria. A nivel de código dimos un salto de calidad: implementamos espectrogramas en escala Mel, probamos una arquitectura sin decodificador (*decoder*), creamos una nueva función de pérdida basada en ventaneo (*windowing*) y programamos un *benchmark* FM. A nivel de documentación, redactamos los avances de los prototipos 1 al 3 y comenzamos a escribir la teoría sobre la escala Mel y los coeficientes MFCC.

- **Fase 5 (Abril - Mayo): Redacción final, entrenamiento de modelos definitivos y cierre**

Hito principal: Finalización de la memoria y entrenamiento de los modelos de producción (P1 a P6).

El desarrollo principal de código finalizó; todo lo programado en estos meses fue para cubrir necesidades de la memoria (métricas de evaluación, *early stopping* para la red y registro del historial de entrenamiento). Nos centramos de lleno en redactar la memoria, elaborar diagramas, limpiar el código, recopilar la bibliografía y preparar los *scripts* para Linux y Windows. Para terminar, entrenamos los seis modelos finales utilizando las máquinas proporcionadas por la UCM.

Capítulo 2

Estado de la Cuestión

2.1. Estado de la cuestión: Audio

2.1.1. Síntesis sonora

La síntesis de sonido es el proceso de generar señales de audio artificialmente mediante hardware electrónico o software, sin depender de la grabación de una fuente acústica real. Las herramientas diseñadas para este propósito se denominan sintetizadores, los cuales generan sonidos en base a unos parámetros modificables.

La base de la síntesis sonora es la onda sinusoidal, descrita por la ecuación fundamental:

$$x(t) = A \sin(2\pi ft + \phi) \quad (2.1)$$

donde A representa la **amplitud** (la altura del sonido, relacionada con el volumen), f es la **frecuencia** en hercios (que determina el tono, con mayores valores produciendo sonidos más agudos), t es el **tiempo**, y ϕ es la **fase inicial** en radianes (que define el punto de inicio de la oscilación). Esta onda de seno pura es el elemento básico de toda síntesis, del cual se generan todos los sonidos.

El **teorema de Fourier** (algfísica) establece que cualquier forma de onda periódica, por compleja que sea, puede descomponerse en una suma de ondas sinusoidales de diferentes frecuencias, amplitudes y fases. Por esta razón, dominar la generación y manipulación de ondas sinusoidales es esencial para comprender cualquier técnica de síntesis sonora.

Históricamente, han existido **diversas aproximaciones a la síntesis**. Las más tradicionales incluyen las ya mencionadas síntesis **aditiva** y **sustractiva**. Estas técnicas fundamentales abrieron el camino hacia métodos matemáticamente más complejos, como la **síntesis FM**.

2.1.2. Síntesis por Modulación de Frecuencia (FM)

La **síntesis FM** es un tipo de síntesis descubierta a finales de los años sesenta (Chowning, 1973). Su funcionamiento se basa en utilizar un oscilador (sistema capaz

de generar una señal que se repite de forma periódica), denominado **modulador** para alterar la frecuencia de otro oscilador, denominado **portador**. El oído humano interpreta esta interacción como la creación de timbres completamente nuevos y armónicamente ricos.

Esta tecnología se popularizó a nivel mundial en 1983 con el lanzamiento del **Yamaha DX7**, el primer sintetizador digital de éxito comercial masivo. Fue muy relevante debido a su capacidad para generar sonidos metálicos, eléctricos y campanados, completamente novedosos para la época.



Figura 2.1: Yamaha DX7. Fuente (Hispanonic).

Sin embargo, la síntesis FM introdujo un problema grave: su **programación es notoriamente contraintuitiva**. Un cambio mínimo en un solo parámetro puede destruir por completo la estructura del sonido. Esta extrema sensibilidad es, precisamente, el principal desafío que justifica la aplicación de redes neuronales para automatizar la inferencia de sus parámetros.

Una formulación simple de la síntesis FM se define como:

$$x(t) = A_c \sin(2\pi f_c t + I \sin(2\pi f_m t)) \quad (2.2)$$

Donde sus componentes principales son:

- A_c : Es la **amplitud del portador**, que define el volumen o ganancia de la señal.
- f_c : Es la **frecuencia portadora**, la frecuencia base del tono que se va a modular.
- f_m : Es la **frecuencia moduladora**, que determina la velocidad a la que oscila la frecuencia del portador.
- I : Es el **índice de modulación**, un factor escalar que controla cuánto se desplaza la fase del portador y, por tanto, la riqueza armónica del sonido resultante.

En nuestro proyecto se implementó una versión más compleja, su ecuación se expande a la siguiente versión, con envolventes¹ de amplitud y modulación:

$$x(t) = A_{env}(t) \sin(2\pi f_c t + I \cdot M_{env}(t) \sin(2\pi f_m t)) \quad (2.3)$$

Donde:

- f_m : Se calcula dinámicamente mediante la relación portadora-moduladora (**ratio**, r), de forma que $f_m = f_c \cdot r$.

¹Una envolvente sirve para controlar la evolución de un sonido en el tiempo(?)

- $A_{env}(t)$: Es la **envolvente de amplitud** (`amp_env`), que sustituye a la constante A_c . Controla la ganancia global de la señal en función del tiempo según las fases de ataque (`a_att`), sostenido (`a_sus`) y decaimiento (`a_dec`).
- $M_{env}(t)$: Es la **envolvente de modulación** (`mod_env`). Escala dinámicamente el índice de modulación en el tiempo entre 0 y 1 a través de sus fases de ataque (`m_att`) y decaimiento (`m_dec`).

2.1.3. Representación digital del sonido: La onda como vector

El sonido en el mundo físico es una onda continua producida por variaciones de presión en el aire. Sin embargo, para que una máquina pueda trabajar con audio, necesita “fotografiar” esa onda miles de veces por segundo en un proceso conocido como **muestreo** (sampling). Como resultado, la onda acústica se transforma en un vector numérico unidimensional. Cada uno de estos valores representa la amplitud de la señal en cada instante de tiempo. El número de muestras capturadas por segundo determina la calidad del audio, esto se denomina **frecuencia de muestreo** (`sample rate`).

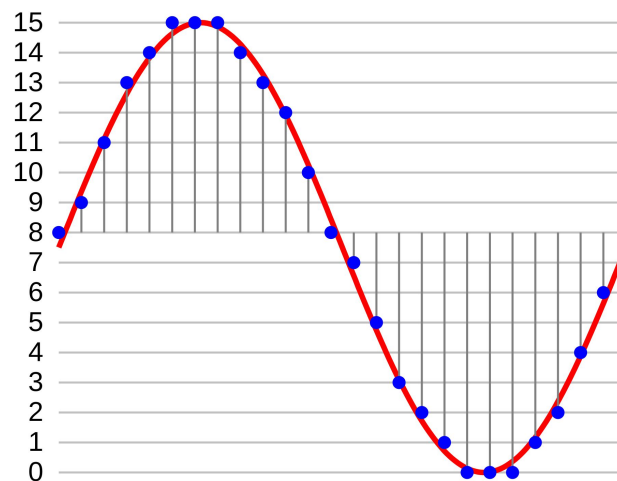


Figura 2.2: Representación del muestreo de una onda. Fuente: Wikipedia (a).

Esta representación es uno de los estándares en tareas de **procesamiento de señales digitales (DSP)**. Sin embargo, existen otras representaciones, como el espectrograma.

2.1.4. Representación espectral: STFT y Espectrogramas

El **espectrograma** es la representación visual del sonido más utilizada en el aprendizaje profundo, ya que permite tratar el audio como una imagen, donde el eje X es el tiempo, el Y la frecuencia y el color la intensidad. En la síntesis FM, esta representación es fundamental para que los modelos identifiquen texturas tímbricas complejas (Yee-King et al., 2018).

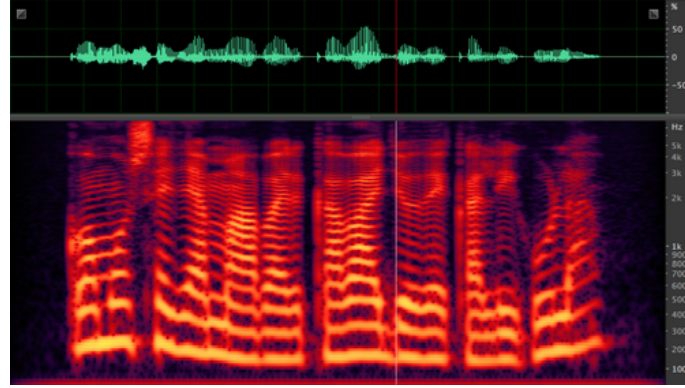


Figura 2.3: Ejemplo de un espectrograma. Fuente: Wikipedia (b).

En la parte superior de la **Figura 2.3** puede verse la forma de onda del sonido, que es la representación unidimensional del vector de audio. En la parte inferior se muestra el espectrograma, que es la representación 2D del mismo sonido.

Esta imagen se genera aplicando la **Transformada de Fourier de Tiempo Reducido (STFT)** sobre el vector de audio. El proceso consiste en dividir la señal en fragmentos cortos mediante una ventana temporal y extraer el contenido frecuencial de cada uno (Allen, 1977). De este modo, se transforma una señal unidimensional en una estructura 2D que conserva la evolución temporal de las frecuencias, facilitando el análisis mediante redes neuronales convolucionales.

2.1.5. Escala de Mel

El eje de frecuencias (Y) en los espectrogramas suele tratarse como una escala lineal, donde los saltos entre frecuencias son uniformes. Sin embargo, esto puede generar una representación no intuitiva para la percepción humana, pues nosotros percibimos el sonido de forma no lineal, para ello se utiliza la **escala Mel**, la cual establece un umbral en una frecuencia específica (700 Hz), a partir de la cual la escala pasa de ser lineal a logarítmica.

Es un sistema de unidades diseñado para que distancias iguales en la escala correspondan a variaciones perceptuales uniformes en el tono (Stevens et al., 1937). Su relevancia radica en que el sistema auditivo posee una **mayor capacidad de percepción en las frecuencias bajas**.

La fórmula de conversión (2.4) es esencial para procesar sonidos de sintetizadores FM, los cuales generan una gran densidad de armónicos superiores que, en una escala lineal, ocuparían la mayor parte del espectro sin aportar información crítica para el oído humano.

$$m = 2595 \cdot \log_{10}(1 + f/700) \quad (2.4)$$

2.1.6. *Mel Frequency Cepstral Coefficients* (MFCC)

Los MFCC son un conjunto de coeficientes que representan la “huella” tímbrica de un sonido, es decir, aíslan el instrumento de la nota que este interpreta. Se obtienen

aplicando la **Transformada de Coseno Discreta (DCT)** al espectrograma Mel para simplificar y organizar la información de las frecuencias (Davis y Mermelstein, 1980).

Si bien fueron el estándar en reconocimiento de voz, la tecnología actual prefiere el uso del espectrograma Mel completo. Este cambio se debe a que las redes neuronales convolucionales (CNN) logran mayor precisión al analizar la imagen sonora total, aprovechando detalles y matices que la compresión de la DCT suele eliminar en el proceso. Sin embargo, **como métrica de evaluación**, los MFCC siguen siendo útiles para comparar la similitud tímbrica entre el sonido original y el resintetizado.

2.2. Estado de la cuestión: IA aplicada al audio

2.2.1. CNNs en el dominio del audio

Las **Redes Neuronales Convolucionales (CNNs)** son un tipo de arquitectura de aprendizaje profundo diseñada para explotar la estructura espacial de los datos, es precisamente por esto que se usan tanto en tareas de imagen. Fueron propuestas originalmente por (LeCun et al., 1998) para el reconocimiento de caracteres escritos a mano. Su principio fundamental es el uso de filtros convolucionales de pequeño tamaño (**kernels**), dichos filtros recorren la entrada realizando productos escalares. Esto les permite detectar patrones sin importar dónde estén ubicados y usar muchos menos parámetros que una red tradicional.

La arquitectura típica alterna **capas convolucionales**, que extraen características locales, con **capas de pooling**, que reducen la dimensionalidad, construyendo así una jerarquía de características que va desde elementos simples, como bordes o gradientes, hacia representaciones abstractas de formas y texturas (Krizhevsky et al., 2012).

En el dominio del audio, las CNNs son eficaces porque pueden aprender patrones locales en el espectrograma, como la estructura armónica y el ruido de ataque. Se ha demostrado que modelos entrenados con grandes cantidades de datos pueden aplicar su conocimiento general a tareas específicas de síntesis (Kong et al., 2020). Al aplicar filtros convolucionales sobre las dimensiones de tiempo y frecuencia, la red identifica texturas tímbricas con una eficiencia muy superior a las redes densas tradicionales.

2.2.2. Transformers en el dominio del audio (AST)

El estado del arte en el procesamiento de audio ha evolucionado de las arquitecturas convolucionales (CNN) hacia los **Transformers**, tras su éxito en el procesamiento de lenguaje natural (Vaswani et al., 2017). En el contexto del audio, el modelo de referencia es el Audio Spectrogram Transformer (AST) (Gong et al., 2021), que adapta la arquitectura de atención para trabajar directamente con espectrogramas. A diferencia de las CNN, que analizan patrones locales mediante filtros, el AST utiliza un mecanismo de auto-atención (**self-attention**). Este permite al modelo capturar relaciones globales y dependencias de largo alcance en el tiempo y

la frecuencia, dividiendo el espectrograma en bloques de datos.

Sin embargo, los Transformers presentan una complejidad computacional cuadrática y requieren volúmenes masivos de datos para superar a las CNN. Por ello, en tareas de sound matching con recursos limitados, como es nuestro caso, **las arquitecturas convolucionales siguen siendo una opción preferente por su eficiencia.**

2.2.3. Inferencia automática de parámetros en síntesis

El **diseño automático de sonido** busca encontrar los parámetros de un sintetizador que repliquen un audio objetivo. Los primeros enfoques exitosos con redes neuronales profundas utilizaron arquitecturas LSTM y CNN para mapear sonidos a las configuraciones de operadores de un Yamaha DX7 (Yee-King et al., 2018). Sin embargo, un hallazgo crítico transformó esta metodología: **optimizar el modelo basándose exclusivamente en la distancia numérica entre parámetros es ineficiente.**

Este fenómeno se debe a la **falta de inyectividad** del motor de síntesis, donde combinaciones de parámetros drásticamente diferentes pueden generar resultados acústicos idénticos. Para solventar esta ambigüedad, se propuso el uso de funciones de pérdida basadas en la distancia entre los espectrogramas del audio original y el resintetizado (**pérdida espectral**) (Barkan et al., 2019). Esta evolución ha convergido en el uso de decodificadores avanzados, **sintetizadores diferenciables** como DDSP (Engel et al., 2020) y arquitecturas basadas en Transformers, que permiten una convergencia mucho más precisa pues priorizan la **coherencia perceptiva** del sonido sobre la coincidencia de valores numéricos.

En nuestro proyecto, las 2 funciones de pérdida que planteamos están directamente inspiradas por estos papers, una con la convergencia espectral y otra con una simulación del ventaneo de la señal que usan en el paper de DDSP.

2.2.4. Representación del audio usada en otros proyectos

El debate sobre la representación óptima divide a la comunidad entre el uso de **características extraídas** (como el espectrograma) y el procesamiento directo de la **onda cruda** (*raw audio*). En el paradigma **End-to-End (E2E)**, se elimina cualquier preprocesamiento analítico, permitiendo que la red reciba la señal unidimensional directamente. Aunque este enfoque exige una enorme capacidad computacional y de memoria debido a la extrema dimensionalidad del audio estándar (por ejemplo, 44 100 muestras por cada segundo), los modelos *End-to-End* son capaces de capturar micro-detalles temporales y relaciones de fase que inevitablemente se pierden al utilizar espectrogramas (van den Oord et al., 2016).

De nuevo, el **Procesamiento de Señales Digitales Diferenciable (DDSP)**, permitiendo que el modelo controle osciladores y filtros directamente, es un muy buen punto medio (Engel et al., 2020).

2.2.5. Datasets

Para entrenar modelos de inferencia eficaces, se requieren bases de datos masivas y rigurosamente etiquetadas. Además, en el contexto del aprendizaje profundo aplicado al audio, la calidad de la muestra es un factor crítico; en representaciones bidimensionales como el espectrograma, la presencia de ruido o artefactos puede provocar que la **Red Neuronal Convolutiva (CNN)** asimile patrones erróneos durante el entrenamiento.

En el ámbito general del análisis musical, el *dataset* **NSynth** se ha consolidado como el estándar actual, proporcionando más de 300 000 notas individuales de diversos instrumentos con etiquetas detalladas sobre sus cualidades acústicas (Engel et al., 2017). Sin embargo, para proyectos centrados específicamente en síntesis FM, es habitual recurrir a la **generación sintética masiva** utilizando emuladores por *software* como *Dexed* (un clon digital del **Yamaha DX7**). Este enfoque garantiza una correspondencia matemáticamente exacta entre el audio exportado y los parámetros internos del sintetizador (Yee-King et al., 2018).

No obstante, en este proyecto, con el objetivo de acotar el problema, se ha optado por desarrollar un motor de síntesis paramétrico propio, implementado íntegramente mediante la librería **NumPy**.



Figura 2.4: Foto del VST Dexed. Fuente: Green Musicians.

Capítulo 3

Metodología y Diseño del Sistema

3.1. Generación del Dataset

FALTA MENCIONAR LAS PRUEBAS INICIALES CON PYO El sistema genera un número (N) de muestras de audio con valores continuos aleatorios, en base a unos rangos de valores específicos para cada parámetro. Para cada uno de ellos se utiliza una distribución uniforme, a excepción de la frecuencia portadora (*Carrier*) que se realiza en escala logarítmica. De esta forma se simula la percepción humana del sonido.

Se sintetizan los N archivos *.wav* mediante la librería **NumPy** y la formulación matemática de la FM (Fase 1). Se define una frecuencia de muestreo (*sample rate*) de 44100 Hz y se incrementa el tiempo de duración de 1 a 2 segundos, con el fin de asegurar un margen temporal necesario para analizar y percibir las variaciones en el sonido provocadas por las envolventes. Estas envolventes se generan sobre un vector de tiempo unificado.

3.2. Preprocesamiento de Datos: normalizaciones y espectrogramas

Una vez generado el dataset, este debe ser preprocesado para que la red lo pueda manejar correctamente. Así pues, los audios *.wav* se convierten a tensores nativos de *PyTorch*. Se cargan mediante *torchaudio* y se les aplica un suavizado de amplitud en los extremos (*fade-in* y *fade-out*) con el objetivo de evitar chasquidos o ruidos indeseados. Tras una normalización de pico, se aplican las dos transformaciones principales: de audio a espectrograma y de amplitud lineal a logarítmica (dB). Para la representación acústica, el programa permite elegir entre dos métodos, con el fin de llevar a cabo diferentes pruebas de experimentación. Por un lado, se encuentran los espectrogramas en formato lineal, mediante **STFT** con 513 *bins*¹ de frecuencia.

¹El término *bin* hace referencia los intervalos de la Transformada de Fourier (FFT). El tamaño de ventana utilizado para la creación de los espectrogramas es de 1024 ($N_{fft} = 1024$), lo que genera $1024/2 + 1 = 513$ intervalos o *bins* de frecuencias

Y por el otro lado, se da lugar a elegir un formato logarítmico-perceptual (**Mel** con 128 bandas), el cual tiene un mayor potencial para la representación del sonido de manera similar a la percibida por el humano.

Posteriormente, se extrae la magnitud de la señal, se convierte a decibelios (dB) y se guarda como tensores (formato *.pt*). Además, se genera un archivo (*spec_mode.txt*) que indica el tipo de espectrograma utilizado.

Para terminar con el tratamiento del dataset, se estructuran los tensores en una clase Dataset de PyTorch (*SpectrogramTensorDataset*). Se lee el registro de etiquetas (*labels.csv*), el cual contiene todos los valores de los parámetros de cada audio, y se calcula la media y desviación estándar global de cada uno de ellos. Con estas estadísticas se aplica una normalización de puntuación estándar (*Z-score*), nivelando así las diferencias entre las escalas de los parámetros, consiguiendo igualar las importancias de cada uno en el cálculo del gradiente. Finalmente, el dataset se divide, mediante una semilla fija, en: 70 % para entrenamiento, 15 % validación y se guarda otro 15 % como conjunto de pruebas (*Test set*).

3.3. Diseño de la Arquitectura CNN

Con el fin de experimentar con distintos modelos en el entrenamiento, se proponen dos arquitecturas distintas.

- **CNNRegressorSimple:** Está formada por un *Encoder* de tres bloques convolucionales, un *Bottleneck* y una cabeza de regresión pura (*Global Average Pooling*).

Primero se pasa por un Codificador (**Encoder**), el cual consta de una secuencia de tres bloques convolucionales que convierten el espectrograma inicial en una representación comprimida, extrayendo así sus características fundamentales. Para ello, cada bloque aplica núcleos (kernels) de 3x3. Se pasa por una capa de normalización por lotes (Batch Normalization), que estabiliza y acelera el entrenamiento, y una activación no lineal ReLU. Finalmente, se aplica una reducción espacial en cada bloque (Max Pooling). Con todo esto, la matriz se ha reducido a una octava parte de su tamaño original y la profundidad de la red ha incrementado de 1 a 128 canales. Como resultado, se obtiene una representación comprimida fiel al sonido original.

Al salir del **Encoder**, el flujo de datos atraviesa lo que se denomina como cuello de botella (**Bottleneck**). Este bloque convolucional no aplica una reducción espacial, manteniendo estables las dimensiones de la matriz. Su función es duplicar el número de canales (profundidad del tensor), pasando de 128 a 256 canales. Esto se hace con la finalidad de mezclar los patrones que el **Encoder** ha extraído previamente, creando una representación densa y unificada.

Después se pasa por la fase de Inferencia Numérica, donde ocurre la regresión de los parámetros del sintetizador FM. Se aplica una capa de agrupamiento global (Adaptive Average Pooling) ya que los parámetros describen el sonido completo y no un punto temporal concreto en el espectrograma. Para ello, se calcula la media de todas las posiciones de cada canal, quedándose con un

único número como resumen por canal. Finalmente, el vector unidimensional, resultado del proceso anterior, atraviesa un **Perceptrón Multicapa**, que lo proyecta hacia las 8 neuronas de salida.

A pesar de que la red ejecuta un entrenamiento rápido, al carecer de reconstrucción no se garantiza que el *Encoder* esté extrayendo de manera acertada la estructura del espectrograma, prediciendo únicamente números.

- **CNNRegressor5**: Se trata de una expansión del modelo anterior, incorporando una segunda cabeza de predicción. Se añade la fase de Reconstrucción (**Decoder**), la cual se encarga de reconstruir el espectrograma original a partir de la representación comprimida del **Bottleneck**. Como el **Encoder** comprime tres veces, el **Decoder** descomprime tres veces. Para ello, utiliza tres capas convolucionales transpuestas (*ConvTranspose2d*).

El motivo para utilizar el **Decoder** es como método de regularización forzada del **Encoder**. En el caso en el que el codificador borre información importante, como armónicos o patrones temporales, que no afecte directamente a los parámetros de la regresión, el **Decoder** no será capaz de redibujar correctamente el espectrograma, disparando así el error en la función de pérdida híbrida. Aparte, si el **Bottleneck** no guarda suficiente información, ocurrirá lo mismo. Todo esto fuerza, mediante el cálculo de gradientes, a que las capas iniciales aprendan a conservar el timbre y la representación del sonido.

3.4. Funciones de Pérdida

FALTA HABLAR DE LOS PAPERS

3.4.1. El problema de la inyectividad

La búsqueda de una función de pérdida se vio totalmente condicionada por una característica esencial de la síntesis FM: su inyectividad es nula. Esto quiere decir que diferentes combinaciones de parámetros pueden dar lugar a sonidos idénticos.

Una red implementada con una función de pérdida que trate de predecir los parámetros exactos del audio original, puede verse afectada negativamente por esta característica, cosa que se comprobó en un primer prototipo cuya pérdida se media de esta forma utilizando el Error Cuadrático Medio (*MSELoss*). Un ejemplo que sufría este modelo, radical pero que explica muy bien este resultado, es un caso en el que el índice de modulación sea 0. En ese caso no es verdaderamente relevante el valor del ratio, sin embargo, la función de pérdida, implementada con *MSE*, penaliza gravemente que este difiera del original.

Otro ejemplo, más común, ocurre en los casos en los que se trabaja sobre un índice de modulación alto, de tal forma que la frecuencia moduladora predomina sobre la portadora. Si tenemos un *Carrier* de 1000 Hz y un *Ratio* de 1.0, la frecuencia moduladora resultante será de 1000 Hz, generándose los armónicos en saltos de mil en mil hercios. Por otro lado, con un *Carrier* de 2000 Hz y un *Ratio* de 0.5, se obtiene la misma moduladora de 1000 Hz y, por lo tanto, los mismos armónicos.

Todo esto sumado a un índice de modulación alto, harían que estos dos audios, paramétricamente muy distantes, suenen muy similares al oído humano.

3.4.2. Funciones

Manteniendo el objetivo de ampliar el espacio de pruebas, se permite también el uso de diferentes funciones de pérdida.

El modelo simple, evalúa la diferencia paramétrica únicamente mediante **SmoothL1 Loss**. Esta se utiliza por su robustez y versatilidad en la regresión numérica: cuando la diferencia entre el valor real y el predicho es muy pequeña, se comporta como un Error Cuadrático (L2), lo que suaviza la convergencia de gradientes; y ante desviaciones grandes, se asemeja al Error Absoluto (L1), de forma que los casos aislados que se alejan del resto no desestabilicen la red. Esta función de pérdida tiene la desventaja de no tener en cuenta la no-inyectividad de la síntesis FM.

En cuanto al modelo CNNRegressor5, se permite la elección de una de las siguientes funciones de pérdida:

- **HybridLoss:** Es una función de pérdida híbrida, que combina el Error Absoluto Medio (L1) de la reconstrucción del espectrograma, un refuerzo estructural de Convergencia Espectral (SC), y la penalización paramétrica anterior. La primera se lleva el peso principal (de 1.0), calcula el Error Absoluto Medio píxel a píxel entre el espectrograma original y el reconstruido (en decibelios). La Convergencia Espectral (SC) se aplica con un peso medio (de 0.5) para evitar los espectros borrosos generados por L1. Esta métrica calcula la norma de Frobenius para evaluar la energía global (relación armónica), regularizando el timbre. A la penalización paramétrica se le da una importancia muy baja (0.05), ya que no es demasiado importante debido al carácter no inyectivo de la síntesis FM.
- **MultiScaleSpectralLoss:** Esta función está inspirada en el modelo de síntesis diferencial DDSP. Al estar utilizando espectrogramas, se simulan los distintos tamaños de ventana de Fourier mediante agrupamientos espaciales temporales (*Average Pooling*). Se agrupa en seis factores (de 1, 2, 4, 8, 16 y 32) y se mide el error en cada una de esas escalas, siendo la media aritmética de esos errores el valor de pérdida total. El cálculo del error de cada escala primero se calcula la diferencia sobre las magnitudes lineales, la cual es útil para capturar picos de energía que pueden ser breves. A esto se le suma después la diferencia sobre las magnitudes en decibelios (logarítmico), multiplicada por un factor α de 1.0, por lo que se le está dando la misma relevancia que el error lineal, capturando así perfectamente la envolvente global y los momentos de baja energía. Mediante esta función, se consigue comparar el sonido obtenido del original en varios niveles de resolución simultáneamente. Por último, se le suma el error paramétrico (*SmoothL1*) con un peso muy bajo (0.05) con el objetivo de que la red no deje de tener mínimamente en cuenta los valores de los parámetros.

3.4.3. Modelo de Pruebas simplificado

Se decide mantener un modelo simplificado, para la realización de pruebas, que infiere únicamente 3 parámetros: frecuencia portadora (*carrier*), frecuencia moduladora (*ratio*) e índice de modulación (*index*).

La topología de la red (*CNNRegressor4*) es igual a la compleja del modelo principal (*CNNRegressor5*), solo que la salida son 3 parámetros en vez de 8. La función de pérdida es la *HybridLoss* explicada en el apartado anterior.

Implementación

4.1. Herramientas y Librerías: *stack* tecnológico

4.2. *Pipeline* de Entrenamiento

Se adjunta la Figura 4.1 con el objetivo de facilitar la comprensión del modelo del Prototipo 5. El trabajo queda dividido en 5 fases: generación de audio, conversión a tensores, preparación del dataset, entrenamiento (con múltiples opciones de arquitectura) e inferencia paramétrica.

Como se muestra en el diagrama 4.1, en primer lugar se encuentra la generación y tratamiento del dataset. La generación se lleva a cabo de la manera especificada en el punto 3.1 y se preprocesa (3.2). Tras esto, el dataset ya está preparado y dividido de tal forma que se tiene un conjunto de entrenamiento que se utilizará en la siguiente fase.

La fase de entrenamiento (fase 4 del diagrama 4.1) comienza con la elección de la arquitectura y función de pérdidas (descritas en 3.3 y 3.4 respectivamente). Y, en la etapa final, se encuentra el algoritmo de retropropagación (*Backpropagation*), mediante el cual se actualizan los pesos para así minimizar el error en la red, calculando los gradientes del error con respecto a cada nodo. Para la actualización de los pesos se emplea el optimizador **Adam** (*Adaptive Moment Estimation*). Este se elige frente al descenso de gradiente común (SGD) debido al cálculo que hace de las tasas de aprendizaje adaptativas e independientes para cada parámetro. A parte, se guardan los pesos de la época que ha conseguido el mínimo histórico con respecto al error sobre el conjunto de validación. Estos valores se almacenan en un archivo de punto de control (checkpoint.pth) junto con el resto de información de contexto necesaria:

- **Parámetros de Desnormalización:** Contiene las medias y desviaciones estándar obtenidas en la Fase 3 del diagrama sobre los parámetros. Estas son necesarias para poder revertir el Z-score en las futuras predicciones.
- **Contexto de Arquitectura:** Son las etiquetas que describen la topología (*arch*: simple o full) y el tipo de espectrogramas (*spec_mode*: stft o mel) que

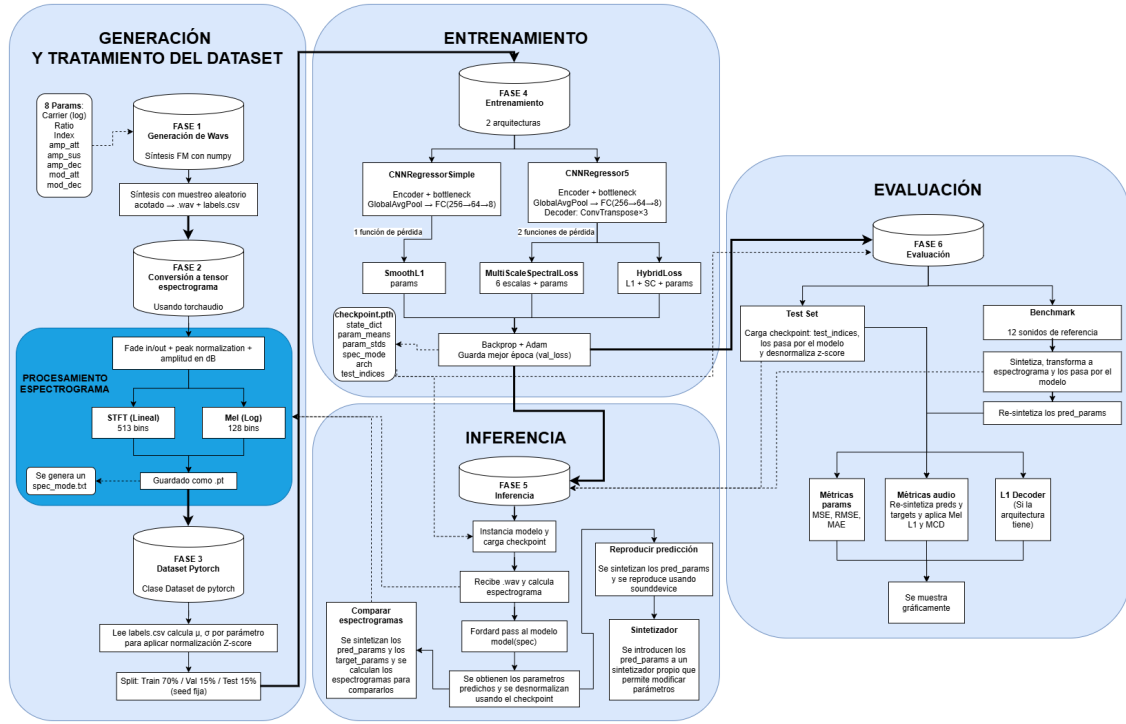


Figura 4.1: Flujo de trabajo del Modelo: desde la síntesis matemática hasta la inferencia y evaluación del modelo.

se han utilizado para el modelo.

- **Aislamiento de Datos:** Contiene los índices del conjunto de prueba (`test_indices`), necesario para la evaluación.

Una vez terminados todos estos procesos, el modelo ya se encuentra entrenado y listo para inferir parámetros o ser evaluado.

4.3. Inferencia de parámetros

La fase 5 (del diagrama anterior) consiste en la inferencia de parámetros de síntesis FM, mediante el modelo entrenado previamente, a partir de audios.

Al inicio de esta fase, se carga el punto de control que instancia automáticamente el tipo de red.

La inferencia comienza cuando se recibe un archivo de audio (`.wav`). En ese momento, se sigue el mismo flujo que en la Fase 2, transformando el audio en un tensor de espectrograma (del tipo indicado por `spec_mode`). Este se introduce en el modelo (*forward pass*), obteniendo así un vector latente de 8 valores, sobre el que se aplica una desnormalización (con los valores de Z-score almacenados en el *checkpoint*) que da como resultado la predicción de los parámetros.

En este punto, el sistema te permite llevar a cabo varias acciones diferentes mediante la interfaz gráfica (GUI). Por un lado, se puede realizar una comparación visual del espectrograma del audio predicho con el del audio original. Para ello,

se sintetizan los parámetros obtenidos y los parámetros originales y se calculan sus espectrogramas. También se da la opción de comparar acústicamente el audio original con el sintetizado por los parámetros inferidos (mediante *sounddevice*) y de exportar estos a un sintetizador propio, con la opción de modificarlos manualmente.

4.4. Interfaz Gráfica de la aplicación

Para una mejor experiencia de uso y facilitar el entorno de entrenamientos y pruebas de la aplicación, se ha desarrollado una Interfaz Gráfica. Se trata de una serie de pantallas sencillas, programadas en Python mediante el uso de la librería *Tkinter*.

En la primera pantalla (7.2), se da la opción de elegir un modelo existente o de entrenarlo.

En caso de decidir cargar un modelo, se podrá seleccionar mediante el explorador de archivos. A continuación (7.3), existen dos posibilidades: probar predicciones independientes de audios o realizar una evaluación general. En la primera, se puede cargar un wav y generar una predicción, existiendo la posibilidad de compararlos auditivamente o gráficamente sus espectrogramas (en frecuencia lineal y en logarítmica) y de ejecutar un sintetizador. En la pantalla del sintetizador(7.4), aparecen los parámetros de la predicción y los del audio original (en caso de tenerlos), siendo modificables. Además, el teclado hace la función de teclas del sintetizador con el sonido generado por los parámetros seleccionados.

La sección del entrenamiento (7.5) es la dedicada a seleccionar varios aspectos relevantes, tanto para el tiempo y forma de entrenamiento como para la topología del modelo. Por un lado, se permite elegir si entrenar el modelo en CPU o en GPU, lo que afectará directamente a su tiempo de entrenamiento. Por el otro, hay que decidir que tipo de espectrograma, arquitectura del modelo y función de pérdida (explorados en el apartado anterior) van a ser los que definan el modelo. El siguiente campo de la pantalla incluye la opción de crear el dataset (explicado anteriormente) o cargar uno personalizado. En último lugar se encuentra el apartado de parámetros modificables para el entrenamiento, como son las épocas (*epoch*), la tasa de aprendizaje (*Learning Rate*) y los pesos de los parámetros de las funciones de pérdida.

4.5. Estructura del Código

Para asegurar la mantenibilidad y escalabilidad, el código se estructura mediante el principio de Separación de Responsabilidades (*Separation of Concerns*). Se divide así la aplicación en dos módulos diferentes:

- **Lógica (Backend):** Se encuentra en el archivo *logica.py*. Aquí se agrupan las funciones matemáticas para la síntesis, calculo de métricas, generación de espectrogramas, etc.
- **Vista y Controlador (Frontend):** Archivo *main.py*. Este implementa la interfaz, recibe los *inputs* del usuario y gestiona el flujo de llamadas a la lógica

y los errores.

Adicionalmente, destaca el uso de la Programación Orientada a Objetos (POO) y el encapsulamiento. Estas características se aplican en la aplicación de la siguiente manera:

- **Clase App (*main.py*):** Se encarga de encapsular el estado global. Evita el uso de variables globales manteniendo las variables necesarias instanciadas en esta clase (*self.dataset_obj*, *self.pathModelo*, *self.model_trained*, etc).
- **Clase SpectrogramTensorDataset (*dataset.py*):** Hereda de la clase base de *Pytorch* de *torch.utils.data.Dataset*, teniendo acceso a métodos como el de procesamiento por lotes (*batching*). Asimismo, esta clase se encarga de los cálculos de estadísticas de normalización y mapeo de los archivos, de forma que desde fuera se obtienen los datos ya preparados mediante el método *__getitem__*.
- **Modelos y Funciones de Pérdida:** Como se ha mencionado previamente, existen diferentes arquitecturas y funciones de pérdida y todas ellas heredan de *nn.Modules*. En consecuencia, el código permite que la lógica del entrenamiento sea independiente del modelo exacto que se esté utilizando, lo que se conoce como **Polimorfismo**. Por otro lado, la clase *HybridLoss* encapsula la lógica de la convergencia espectral y la norma de Frobenius, manteniendo el entrenamiento ajeno a estas fórmulas.
- **FMSynth8 Window:** La pantalla del sintetizador se encuentra totalmente desacoplada del resto de la lógica, siendo una clase independiente. De esta manera, se puede utilizar tanto desde la ejecución normal de la aplicación como de forma aislada, facilitando así su construcción, ampliación y pruebas aisladas.

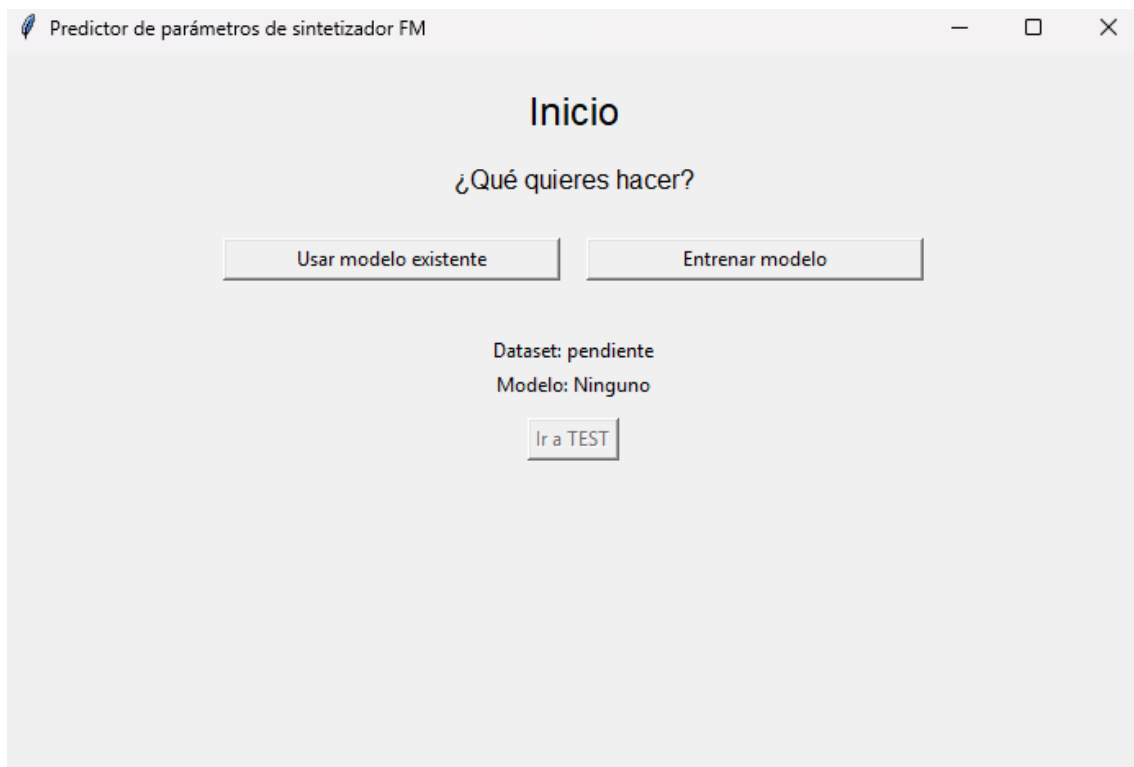


Figura 4.2: Primera pantalla de la Interfaz Gráfica de la Aplicación, elección uso modelo o entrenamiento.

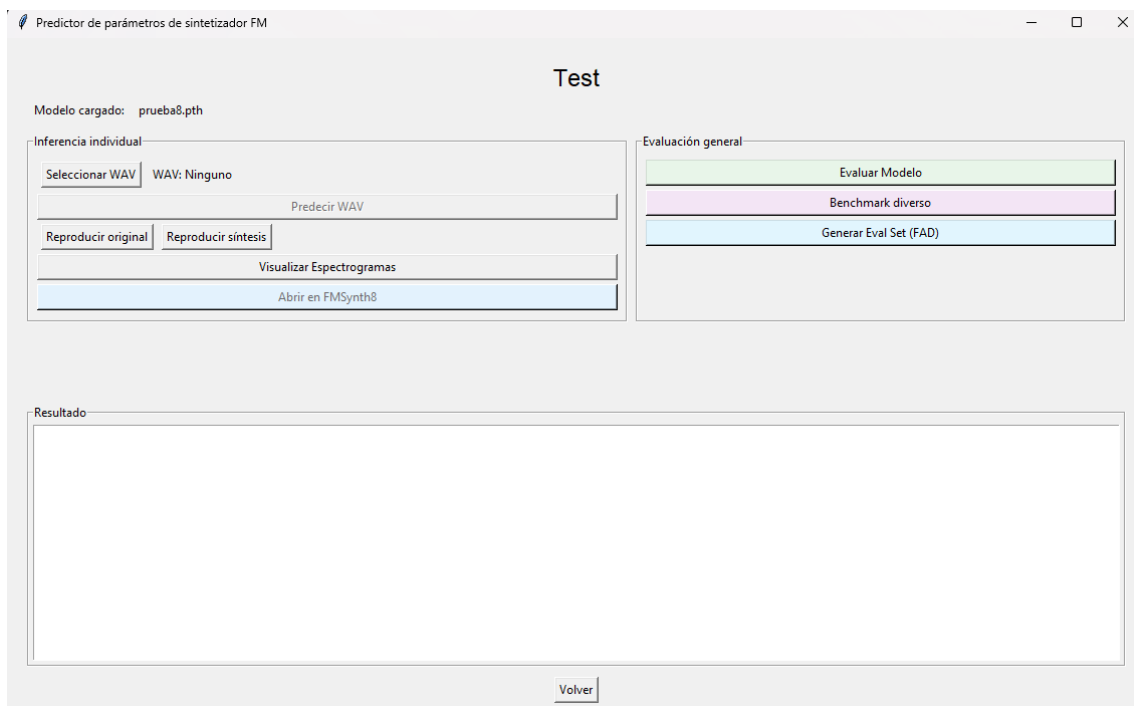


Figura 4.3: Pantalla en la que se pueden hacer pruebas con el modelo seleccionado.

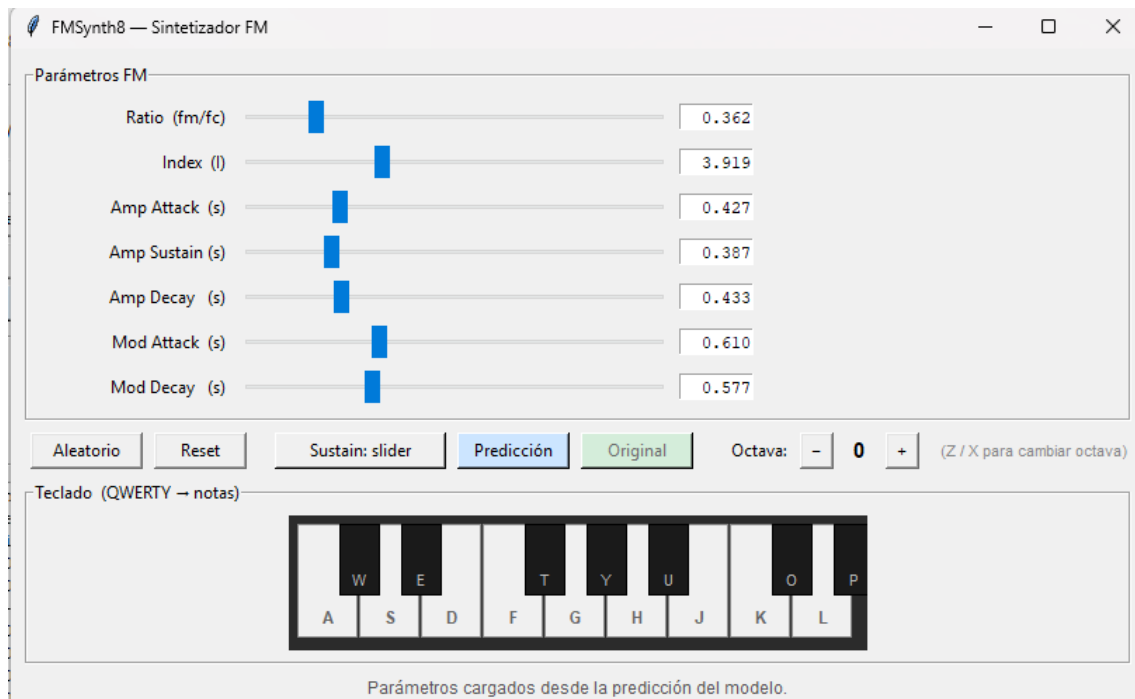


Figura 4.4: Sintetizador FM de prueba.

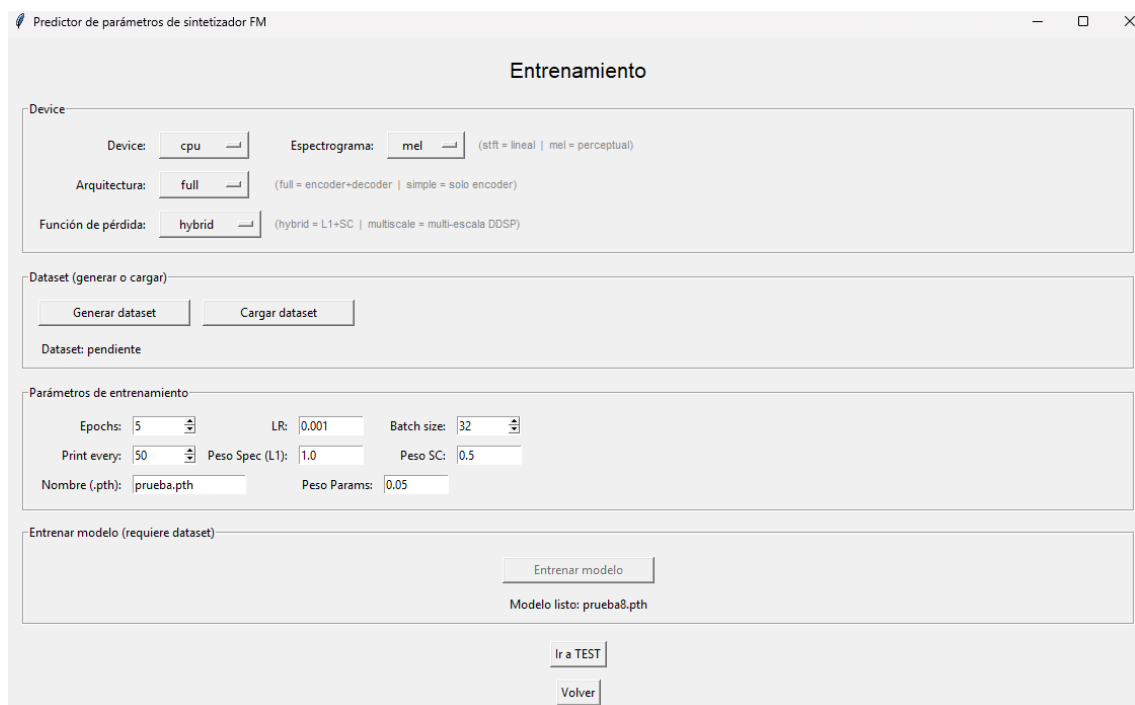


Figura 4.5: Sintetizador FM de prueba.

Capítulo 5

Evolución Arquitectónica

5.1. Exploración de Paradigmas: Clasificación vs. Regresión

5.1.1. Enfoque de Clasificación

La clasificación es el paso previo natural a la búsqueda de un modelo de inferencia de parámetros. De esta manera, se puede comprobar de forma práctica que la red neuronal procesa el audio adecuadamente y es capaz de conservar su información acústica fundamental.

El **Prototipo 1** es una prueba de concepto para la primera aproximación al aprendizaje automático aplicado al audio. En él se utiliza un dataset reducido, construido con la librería NumPy, de alrededor de 50 archivos *.wav* de un segundo de duración. Estos audios se generan sintéticamente con frecuencias aleatorias y formas de onda básicas (*sine*, *square*, *sawtooth*, *triangle* y *noise*).

En la fase de preprocesamiento y entrenamiento, desarrollada utilizando TensorFlow, se opta por no aportar a la red formas de audio crudo. En su lugar, se extraen las características calculando los Coeficientes Cepstrales en las Frecuencias de Mel (MFCC) de cada muestra. El modelo tiene el objetivo de clasificar y asociar la matriz de coeficientes con la etiqueta correspondiente a su tipo de onda. A pesar de esto, al plantear una arquitectura básica no convolucional, la red neuronal carece de los mecanismos para realizar una interpretación correcta de los patrones bidimensionales de los MFCC. Como consecuencia, el modelo arroja una precisión muy baja.

El **Prototipo 2** se basa en el Prototipo 1. Sigue atendiendo al objetivo de la clasificación, pero tiene características que difieren del anterior. Esta vez, los audios se convierten a espectrogramas *.png* mediante el uso de la librería de *librosa*. De esta forma se consigue un menor coste computacional y de almacenamiento. Al tratarse realmente de imágenes, se pueden utilizar modelos preentrenados de visión capaces de clasificarlas.

En el entrenamiento se hace uso de **ResNet34 preentrenada** vía *fastAI*. Para su adaptación, se utilizan **DataBlocks**, una herramienta de *fastAI* mediante la cual

se especifican que las entradas son las imágenes (espectrogramas) y las salidas son las categorías (tipo de onda). El etiquetado de la categoría de cada archivo se obtiene a partir su nombre y se divide destinando un 80 % de las imágenes para entrenar y un 20 % para validación.

Como resultado de utilizar una red convolucional y la arquitectura externa **ResNet34**, el modelo obtuvo una buena precisión, especialmente identificando ondas puras dentro del rango entrenado.

5.1.2. Enfoque de Regresión

La clasificación sirvió como prueba de concepto y como primer acercamiento a un modelo de procesamiento de audio, permitiendo que se explorasen tecnologías, bibliotecas y métodos que serían de gran utilidad durante la totalidad de las fases del desarrollo.

Sin embargo, para la implementación de un modelo que cumpliera con la descripción del trabajo, se decide dar un salto hacia la síntesis paramétrica continua. Para ello, fue necesario tomar una decisión de diseño fundamental: la elección del motor de síntesis. Tras la evaluación de distintas alternativas, se optó por centrar el proyecto en la síntesis por Modulación de Frecuencia (FM). Esta técnica presenta ventajas claras para el aprendizaje automático. A diferencia de la síntesis aditiva, que requiere decenas de osciladores simultáneos, la síntesis FM es muy eficiente, consiguiendo generar timbres complejos a partir de un número reducido de parámetros. De esta forma, se puede comenzar utilizando, por ejemplo, la frecuencia portadora, el ratio y el índice de modulación, y luego ampliar el número de variables para conseguir sonidos más complejos. Estas características acotan la dimensionalidad del problema predictivo en un rango viable para la red neuronal.

Si bien con la clasificación queda validada la extracción de características acústicas, una vez tomada esta última decisión, deja de ser útil. Para la inferencia de parámetros de la síntesis FM, resulta obligatoria la evolución hacia una arquitectura de regresión.

Para el **Prototipo 3** se genera un dataset mediano de aproximadamente 15000 muestras, construido mediante un barrido de parámetros en un sintetizador FM de **pyo**. Para la reducción de la carga computacional y la optimización del espacio, se sustituye **librosa** por la librería de *torchaudio*. Esta permite transformar los archivos de audio (*.wav*) en tensores de espectrogramas (*.pt*), asociados mediante el uso de un (*.csv*) a sus valores numéricos. El 80 % del conjunto de datos se destina a entrenamiento y el otro 20 % a validación.

El entrenamiento se implementa mediante **PyTorch**. Se diseña una arquitectura convolucional propia (SmallCNNRegressor). La red neuronal se compone de dos bloques principales: uno de extracción de características y una cabeza de regresión.

El primer bloque recibe los espectrogramas en forma de tensores de un único canal (lo que equivaldría a imágenes en escala de grises) y los procesa en otros tres bloques convolucionales. Cada uno de estos aplica kernels de tamaño 3x3 (o núcleos convolucionales) que se deslizan sobre la matriz del espectrograma para extraer características espaciales, detectando así patrones visuales en la representación espectral del sonido. A esto le sigue una función de activación no lineal **ReLU** y una

capa de reducción espacial (**Max Pooling**). El último bloque convolucional termina en una capa de agrupamiento (**Adaptive Average Pooling**), que comprime los mapas de características a una matriz 4x4.

El siguiente paso es la cabeza de regresión, en donde la información espacio-temporal del espectrograma (extraída en el bloque anterior), se pasa a un vector de una única dimensión y atraviesa dos capas completamente conectadas (**Perceptrón Multicapa**). La primera, comprime la información a 128 neuronas, y la segunda, saca el vector por una dimensión de salida de tamaño 3, que se corresponde con los tres parámetros del sintetizador: frecuencia portadora, ratio y el índice de modulación.

El modelo optimiza el proceso de los espectrogramas mediante el algoritmo Adam, utilizando el Error Cuadrático Medio (MSELoss) como función de pérdida. Y, en cuanto al ciclo de entrenamiento, este se configuró con una duración de 5 épocas (*epochs*), un número bajo pero suficiente para analizar el comportamiento del modelo. En este prototipo se decidió no hacer una búsqueda extensa de hiperparámetros al tratarse de un prototipo intermedio, con un fin de experimentación inicial con la regresión.

A pesar de esto, se puede observar una precisión muy limitada en los resultados experimentales. Se consigue mantener un MSE bajo en el *Ratio* e *Index*, pero el error en el *Carrier* resulta desproporcionado, debido a operar en la escala de miles de hercios. De este prototipo se puede sacar la conclusión de que minimizar el error matemático sobre los parámetros no garantiza la similitud acústica del sonido generado y de la necesidad de trabajar sobre parámetros normalizados para contrarrestar las grandes diferencias en sus escalas.

5.2. El Problema de la Inyectividad y la Necesidad de una Arquitectura Compleja

El Prototipo 3 contiene una limitación fundamental muy relevante en la síntesis FM: su inyectividad es nula. Esto quiere decir que diferentes combinaciones de parámetros pueden dar lugar a sonidos idénticos. Un ejemplo, radical pero que explica muy bien este resultado, es un caso en el que el índice de modulación sea 0. En ese caso no es verdaderamente relevante el valor del ratio, sin embargo, la función de pérdida, implementada con **MSE**, penaliza gravemente que este difiera del original. Este obstáculo hace que se tenga que replantear la función de pérdida.

Otro ejemplo, más común, ocurre en los casos en los que se trabaja sobre un índice de modulación alto, de tal forma que la frecuencia moduladora predomina sobre la portadora. Si tenemos un *Carrier* de 1000 Hz y un *Ratio* de 1.0, la frecuencia moduladora resultante será de 1000 Hz, generándose los armónicos en saltos de mil en mil hercios. Por otro lado, con un *Carrier* de 2000 Hz y un *Ratio* de 0.5, se obtiene la misma moduladora de 1000 Hz y, por lo tanto, los mismos armónicos. Todo esto sumado a un índice de modulación alto, harían que estos dos audios, paramétricamente muy distantes, suenen muy similares al oído humano.

5.2.1. Prototipo 4: Red Convolutiva de Doble Cabeza

El **Prototipo 4** se diseña para solucionar el problema de la inyectividad, tratando de asegurar que la inferencia de parámetros se corresponda con la fidelidad acústica. Este modelo se basa en una arquitectura convolutiva de doble cabeza (**Encoder-Decoder**).

En cuanto al preprocesamiento del conjunto de datos, se implementa una mejora para evitar que la diferencia tan grande entre las escalas de diferentes parámetros (como la frecuencia portadora frente al ratio) comprometa el cálculo del gradiente. Los tensores se normalizan estadísticamente (**Z-score**), restando la media y dividiendo por la desviación estándar de cada parámetro.

Para cada parámetro x , el valor normalizado z se calcula según la Ecuación 5.1:

$$z = \frac{x - \mu}{\sigma} \quad (5.1)$$

Donde μ representa la media aritmética de dicho parámetro en todo el dataset de entrenamiento y σ su desviación estándar.

La topología de la red (CNNRegressor4), consta de un procesamiento dividido en varias fases:

Primero se pasa por un Codificador (**Encoder**), el cual consta de una secuencia de tres bloques convolutivos que convierten el espectrograma inicial en una representación comprimida, extrayendo así sus características fundamentales. Para ello, cada bloque aplica núcleos (kernels) de 3x3. Posteriormente, pasa por una capa de normalización por lotes (Batch Normalization), que estabiliza y acelera el entrenamiento, y una activación no lineal ReLU. Finalmente, se aplica una reducción espacial en cada bloque (Max Pooling). Con todo esto, la matriz se ha reducido a una octava parte de su tamaño original y la profundidad de la red ha incrementado de 1 a 128 canales. Como resultado, se obtiene una representación comprimida rica en características acústicas.

Al salir del **Encoder**, el flujo de datos atraviesa lo que se denomina como cuello de botella (**Bottleneck**). Este bloque convolutivo no aplica una reducción espacial, manteniendo estables las dimensiones de la matriz. Su función es duplicar el número de canales (profundidad del tensor), pasando de 128 a 256 canales. Esto se hace con la finalidad de mezclar los patrones que el **Encoder** ha extraído previamente, creando una representación densa y unificada.

Después se pasa por la fase de Inferencia Numérica, donde ocurre la regresión de los parámetros del sintetizador FM (frecuencia portadora, ratio e índice). Se aplica una capa de agrupamiento global (Adaptive Average Pooling) ya que los parámetros describen el sonido completo y no un punto temporal concreto en el espectrograma. Para ello, se calcula la media de todas las posiciones de cada canal, quedándose con un único número como resumen por canal. Finalmente, el vector unidimensional, resultado del proceso anterior, atraviesa un **Perceptrón Multicapa**, que lo proyecta hacia las tres neuronas de salida.

Por último, se encuentra la fase de Reconstrucción (**Decoder**), la cual se encarga de reconstruir el espectrograma original a partir de la representación comprimida del **Bottleneck**. Como el **Encoder** comprime tres veces, el **Decoder** descomprime tres veces. Para ello, utiliza tres capas convolutivas transpuestas (ConvTranspose2d).

El motivo para utilizar el **Decoder** es como método de regularización forzada del **Encoder**. En el caso en el que el codificador borre información importante, como armónicos o patrones temporales, que no afecte directamente a los parámetros de la regresión, el (**Decoder**) no será capaz de redibujar correctamente el espectrograma, disparando así el error en la función de pérdida híbrida. Aparte, si el **Bottleneck** no guarda suficiente información, ocurrirá lo mismo. Todo esto fuerza, mediante el cálculo de gradientes, a que las capas iniciales aprendan a conservar el timbre y la representación del sonido.

5.2.2. Prototipo 4: Función de Pérdida Híbrida

Con el objetivo de solucionar el problema de la inyectividad, se consolida el uso de utilizando Función de Pérdida Híbrida, abandonando el Error Cuadrático Medio (MSE). Esta función evalúa la pérdida de dos formas simultáneamente, dándole prioridad a la similitud y convergencia de espectrogramas (pérdida y convergencia espectral) y dejando en segundo plano la similitud de los parámetros (pérdida paramétrica).

Por un lado tenemos la **Pérdida y Convergencia Espectral**. Estas funciones son las principales para la composición del error, teniendo mucho más peso que la pérdida paramétrica, y estando encargadas de asegurar que un sonido generado sea acústicamente similar al original.

La **pérdida espectral** se calcula mediante el Error Cuadrático Medio (**L1 Loss**), el cual mide la diferencia directa entre espectrogramas (píxel a píxel) en escala logarítmica (dB), siendo la métrica de mayor peso para el error. Con esto se garantiza unos picos de energía y formas generales que coincidan en tiempo y frecuencia.

Matemáticamente, si S es el espectrograma original en decibelios y $\hat{S}$ el reconstruido, ambos con N elementos, la pérdida L1 se define como:

$$L_1 = \frac{1}{N} \sum_{i=1}^N |S_i - \hat{S}_i| \quad (5.2)$$

En cuanto a la **convergencia espectral**, es un refuerzo (tiene un peso bajo) para la pérdida espectral, que compara las magnitudes de forma relativa con la norma de Frobenius, capturando la estructura global del espectro (como los armónicos y la energía). Estas características son muy relevantes para encontrar una representación fiel al timbre original.

La convergencia espectral (C_S) emplea la norma de Frobenius ($\|\cdot\|_F$) sobre las magnitudes matriciales para medir la distancia relativa entre las energías, tal y como detalla la Ecuación 5.3:

$$C_S = \frac{\|S - \hat{S}\|_F}{\|S\|_F} \quad (5.3)$$

Por último se encuentra la **Pérdida Paramétrica**, aplicada mediante **SmoothL1**. Esta penaliza las diferencias en los parámetros del sintetizador, por lo que no se elimina totalmente la regresión numérica de los parámetros, pero se le da un peso considerablemente bajo.

La Ecuación 5.4 muestra el comportamiento por tramos de SmoothL1 para una predicción $\hat{y}$ frente a su valor real y :

$$L_{smooth} = \begin{cases} 0,5(y - \hat{y})^2, & \text{si } |y - \hat{y}| < 1 \\ |y - \hat{y}| - 0,5, & \text{en caso contrario} \end{cases} \quad (5.4)$$

La ponderación de los pesos en la función de pérdida híbrida es fundamental para el correcto aprendizaje del Modelo y también para solucionar el problema de la inyectividad. Se le otorga un peso completo (1.0) a la pérdida espectral, un peso medio (0.5) a la convergencia espectral y un peso muy bajo (0.05) a la pérdida paramétrica. De esta manera, en el caso de que la red infiera una serie de parámetros que difieran considerablemente de los parámetros originales, si sus espectrogramas son muy similares (y, por tanto, su timbre también lo es), el error será mínimo. El modelo debe memorizar números exactos, si no que analiza la imagen del sonido, su espectrograma.

Experimentación Previa

6.1. Limitaciones de Fréchet Audio Distance (FAD)

Para la primera evaluación del modelo, se seleccionó la métrica *Fréchet Audio Distance* (FAD), descrita en la Sección ???. Su implementación en el proyecto, se realiza a través de la comparación de dos conjuntos de datos: una muestra de 2.000 audios FM generados aleatoriamente y sus predicciones con el modelo utilizado en el *Prototipo 4*. El resultado obtenido fue de 0.9999, indica una similitud estadística casi absoluta entre las distribuciones de audio.

A pesar de que el resultado obtenido plantea una evaluación teóricamente óptima, el análisis profundo de los resultados y del funcionamiento de FAD indican una discrepancia. El motivo de la poca validez de la métrica en el proyecto, se atribuye a un error en la interpretación de su utilidad real, ya que el FAD está diseñado para evaluar la calidad global de modelos generativos ?, mientras que este proyecto se centra en la estimación de los parámetros que luego utiliza para generar audio FM. Puesto que tanto el conjunto de referencia como las predicciones se crean mediante síntesis FM, su huella espectral y sus características de alto nivel (*embeddings*) son casi idénticas ?. Por tanto, el FAD evalúa positivamente que el modelo ha aprendido a recrear el timbre o "textura" de la síntesis FM, pero, como se comentaba previamente, resulta una muestra ciega a precisión muestra a muestra. En resumen, se prioriza la verosimilitud estadística del sonido sobre la similitud de las muestras una a una ??.

6.2. Validación por Fuerza Bruta (Grid Search)

Una de las cuestiones principales que se plantean en el desarrollo de este proyecto es si el uso de un modelo de Inteligencia Artificial representa la aproximación más óptima para el objetivo de la inferencia de parámetros. Esta disyuntiva surge, sobre todo, al analizar los primeros prototipos, como el *Prototipo 4*, en el que se emplean únicamente 3 parámetros para la síntesis FM: *Carrier*, *Index* y *Ratio*. Debido al reducido tamaño de este espacio de parámetros, resulta lógico plantear en la viabilidad de realizar un método de búsqueda exhaustiva (*Grid Search*). Este enfoque consiste en realizar un recorrido iterativo por todas las combinaciones posibles, evaluándolas

y registrando aquellas configuraciones que presenten una mayor similitud acústica con respecto al sonido objetivo.

Por otro lado, surge la necesidad de encontrar un método robusto para evaluar la fidelidad del modelo de inferencia de parámetros, buscando una métrica que simule la percepción de un oído humano experto haría un oído humano experto. Esta necesidad y la cuestión anterior convergen en la fase de experimentación. De este modo, se consiguen probar diferentes métricas que evalúen la similitud acústica dentro del algoritmo de *Grid Search*, comprobando simultáneamente la viabilidad técnica del enfoque de fuerza bruta frente al modelo predictivo.

Para lograr esta simulación del sistema auditivo humano, la **Escala Mel** se ha consolidado como el estándar en el procesamiento digital de señales de audio. Esta métrica aplica una transformación logarítmica que modela el comportamiento no lineal del oído humano, el cual pierde resolución y sensibilidad con el aumento de la frecuencia.

Por otra parte, para este mismo fin también se contempla el uso de los *Mel Frequency Cepstral Coefficients* (MFCC). Esta métrica aplica una Transformada del Coseno Discreta (DCT) sobre un espectrograma previamente mapeado en la escala Mel. La principal diferencia, donde radica su interés, es la capacidad de aislar la envolvente espectral del sonido, consiguiendo una representación matemática precisa del timbre o la textura acústica del audio. Al comprimir la información en un vector de coeficientes de baja dimensionalidad, MFCC permite calcular eficientemente la distancia matemática entre el sonido objetivo y los sonidos conseguidos en las iteraciones del *Grid Search*.

6.2.1. Diseño experimental y parametrización del Grid Search

El espacio de parámetros del sintetizador FM seleccionado para el algoritmo de *Grid Search*, buscando un balance entre resolución de la búsqueda y coste computacional, fue el siguiente:

- **Carrier (Frecuencia portadora):** Rango de 100 Hz a 2000 Hz, con incrementos de 100 Hz (20 valores).
- **Ratio (Frecuencia moduladora):** Rango de 0.05 a 2.0, con incrementos de 0.05 (40 valores).
- **Index (Índice de modulación):** Rango de 1.0 a 10.0, con incrementos de 0.5 (19 valores).

Esta configuración genera **15.200 combinaciones únicas**. Por cada iteración del triple bucle, el algoritmo crea un audio de un segundo de duración mediante síntesis FM. A continuación, se calcula el Error Cuadrático Medio (MSE) respecto a las características del audio original.

6.2.2. Espectrograma de Mel

La mayor parte de los resultados ejecutados con la métrica Mel en *Grid Search*, muestran valores que difieren significativamente de los originales. Debido a la dis-

cretización del espacio de búsqueda, el algoritmo tiende a alcanzar mínimos locales manipulando parámetros secundarios para aproximar la distribución de energía evaluada por el Espectrograma Mel.

Un ejemplo claro de este comportamiento se observa al analizar una inferencia donde los parámetros objetivo eran $Carrier = 1055,54$ Hz, $Ratio = 0,704$ e $Index = 9,38$. Los resultados de los tres mejores candidatos del *Grid Search* fueron los siguientes:

```
=====
Tiempo total de ejecución: 1 minutos y 5.16 segundos
=====
```

```
#1 -> Error Mel: 113.6648 | Carrier: 400.0, Ratio: 1.85, Index: 9.5
#2 -> Error Mel: 126.5552 | Carrier: 400.0, Ratio: 1.85, Index: 9.0
#3 -> Error Mel: 148.8820 | Carrier: 400.0, Ratio: 1.85, Index: 10.0
```

Aunque la divergencia en la frecuencia portadora ($Carrier$) es superior a 600 Hz, el cálculo de la frecuencia moduladora latente ($f_m = Carrier \times Ratio$) explica esta selección. En el audio objetivo, la frecuencia moduladora es de $1055,54 \times 0,704 \approx 743,07$ Hz. Por otra parte, la frecuencia moduladora en la combinación seleccionada por el algoritmo es de $400,0 \times 1,85 = 740,0$ Hz.

Al operar con un índice de modulación elevado ($Index > 9$), la energía espectral de la frecuencia portadora se atenúa severamente, trasladando el peso acústico a las bandas laterales generadas por la modulación. El algoritmo de fuerza bruta, incapaz de seleccionar el tono original por la limitación de la rejilla, optó por anclar el parámetro $Carrier$ en 400 Hz para forzar un $Ratio$ que replicara con precisión la frecuencia moduladora original (740 Hz). Esto valida empíricamente que, sin un espacio de inferencia continuo como el que proporciona el modelo predictivo de IA, las métricas espectrales son susceptibles a la falsificación de la frecuencia fundamental mediante intermodulación armónica.

El siguiente ejemplo muestra qué ocurre si esta vez se utiliza un $Carrier$ que el algoritmo no pueda seleccionar por la limitación de la rejilla (saltos de 100 Hz) y un índice de modulación bajo.

Parámetros originales:

```
Carrier = 1752.58
Ratio = 0.62
Index = 1.03
```

La aproximación del algoritmo fue la siguiente:

```
=====
Tiempo total de ejecución: 1 minutos y 4.39 segundos
=====
```

```
#1 -> Error Mel: 176.1064 | Carrier: 1600.0, Ratio: 0.70, Index: 2.0
#2 -> Error Mel: 177.4902 | Carrier: 1600.0, Ratio: 0.70, Index: 1.5
#3 -> Error Mel: 189.0954 | Carrier: 1700.0, Ratio: 0.65, Index: 1.0
```

En este escenario de prueba, el algoritmo dejó en tercera posición a la opción con los valores más parecidos a la original Mientras que la tercera opción, al oído resulta muy similar pero desafinada, la primera opción difiere excesivamente.

Matemáticamente, al incrementar artificialmente el *Index*, el algoritmo aumenta la anchura de banda del espectro, "difuminando" la energía a lo largo de múltiples bandas laterales. Este espectro ensanchado logra solaparse parcialmente con la frecuencia objetivo (1752 Hz), minimizando el MSE global en la matriz. Sin embargo, acústicamente, el resultado es catastrófico: el algoritmo ha sacrificado un tono puro y definido por un sonido rico en armónicos y percusivo, únicamente para evadir la penalización de un mínimo local inaccesible.

6.2.3. Espectrograma MFCC

En contraposición a lo observado con el Espectrograma Mel, la evaluación mediante los Coeficientes Cepstrales en las Frecuencias de Mel (MFCC) obtiene unos resultados lo más cercanos posibles a los valores iniciales de los parámetros, teniendo en cuenta que la anchura de la rejilla impide que sean más exactos.

Por esta razón, el audio obtenido queda, en la mayoría de ocasiones, notablemente desafinado con respecto al original. Sin embargo, se consigue plasmar la *textura* (o timbre) del sonido de una manera prácticamente idéntica a la original, debido a la precisión en la inferencia de los parámetros de *Ratio* e *Index*.

Los siguientes resultados son los obtenidos con los mismo audios que los utilizados de ejemplo en Mel.

Parámetros originales:

Carrier = 1055.54

Ratio = 0.704

Index = 9.38

Aproximación del algoritmo:

```
=====
Tiempo total de ejecución: 1 minutos y 10.58 segundos
=====
```

```
#1 -> Error MFCC: 124.7339 | Carrier: 1100.0, Ratio: 0.70, Index: 9.0
#2 -> Error MFCC: 146.7065 | Carrier: 1100.0, Ratio: 0.70, Index: 8.5
#3 -> Error MFCC: 171.0587 | Carrier: 1100.0, Ratio: 0.65, Index: 9.5
```

Parámetros originales:

Carrier = 1752.58

Ratio = 0.62

Index = 1.03

La aproximación del algoritmo fue la siguiente:

```
=====
Tiempo total de ejecución: 1 minutos y 9.93 segundos
=====
```

```
#1 -> Error MFCC: 121.4236 | Carrier: 1700.0, Ratio: 0.70, Index: 1.0
#2 -> Error MFCC: 143.8489 | Carrier: 1700.0, Ratio: 0.65, Index: 1.0
#3 -> Error MFCC: 155.1412 | Carrier: 1800.0, Ratio: 0.65, Index: 1.5
```

En estos resultados se puede apreciar lo teorizado previamente, el algoritmo encuentra los valores más cercanos a los originales dentro de los saltos de la rejilla. Aun así, cabe mencionar que MFCC no es tan sensible a los cambios de *Pitch* como lo es Mel, lo que implica que, aun teniendo unos saltos minúsculos en el *Carrier*, MFCC no conseguiría en todas las ocasiones la nota exacta.

6.2.4. Conclusiones

El análisis de los datos obtenidos en estas pruebas ha servido, por un lado, para responder a las cuestiones iniciales y, por otro, ha planteado nuevas ideas y preguntas.

En cuanto a la viabilidad del método *Grid Search* para la inferencia de parámetros de un sintetizador FM, se demuestra con contundencia que los tiempos de ejecución (que aparecen en los resultados anteriores) de dicho algoritmo son órdenes de magnitud superiores a los obtenidos en la síntesis realizada por el modelo predictivo. Teniendo el primero una media aproximada de 1 minuto y 5 segundos para Mel y 1 minuto y 10 segundos para MFCC, frente a los 2 o 3 segundos que tarda el modelo de inferencia. A esto se le añade el hecho de que se está utilizando un tamaño de rejilla bajo de únicamente 15200 iteraciones, si se ampliaran estos números los tiempos aumentarían también.

Aparte de los tiempos de ejecución, cabe destacar la precisión. Mientras que el método *Grid Search* realiza una búsqueda discreta de valores, que está sujeta al tamaño de rejilla, la red neuronal es capaz de operar en un espacio continuo.

La siguiente cuestión planteada era la búsqueda de un método de evaluación del modelo de inferencia. Con los análisis llevados a cabo se ha podido comparar entre la opción de utilizar *Espectrogramas de Mel* o *Espectrogramas MFCC*. Se ha observado que, al utilizar la métrica obtenida con *Mel*, se van a evaluar de manera positiva combinaciones de parámetros con diferente *Index* que mantengan una energía espectral similar, sin importar de forma estricta si esta proviene de la frecuencia portadora o de las bandas laterales moduladoras. Esto deriva en cambios significativos en el timbre de la predicción frente al audio original. Por otro lado, la métrica obtenida con *MFCC* presenta una gran sensibilidad frente a los cambios en el timbre, lo que resulta de gran utilidad para la evaluación del modelo.

Por último, se deja planteada la idea de utilizar una función de pérdida híbrida que utilice tanto *Mel* como *MFCC*. La razón para llevar dicha implementación a cabo es que se ha visto, durante el proceso de investigación para las anteriores pruebas, ambos son el *estado del arte* en lo que a modelos predictivos de audio se refiere y se ha considerado que con una implementación híbrida con ellos, se podría extraer la

ventaja de encontrar el *Pitch* de *Mel* y la de encontrar el timbre de *MFCC*.

Arquitectura Final

Se adjunta la Figura 7.1 con el objetivo de facilitar la comprensión del modelo del Prototipo 5. El trabajo queda dividido en 5 fases: generación de audio, conversión a tensores, preparación del dataset, entrenamiento (con múltiples opciones de arquitectura) e inferencia paramétrica.

7.1. Expansión Paramétrica: Evolventes y Generación Aleatoria del Dataset

Las predicciones del **Prototipo 4** consiguen replicar detalladamente el timbre. Sin embargo, los sonidos generados en esta etapa están muy alejados de las aplicaciones reales del **Sound Matching**. Por esta razón, se propone un nuevo modelo en el que se explora ampliar la dimensionalidad de 3 a 8 parámetros. A las variables fundamentales (frecuencia portadora, ratio e índice), se le añaden las envolventes temporales: la envolvente de amplitud (*attack*, *decay* y *sustain*) y la envolvente del índice de modulación (*attack* y *decay*).

Son precisamente estas características del **Prototipo 5**, por las que es necesario una nueva forma de generación del dataset. La ampliación de 3 a 8 parámetros hace computacional inviable generar el dataset mediante un barrido de combinaciones posibles. Para dar solución a esta limitación, se genera el dataset por muestreo aleatorio.

7.1.1. Generación y Tratamiento del Dataset

El sistema genera un número (N) de muestras de audio con valores continuos aleatorios, en base a unos rangos de valores específicos para cada parámetro. Para cada uno de ellos se utiliza una distribución uniforme, a excepción de la frecuencia portadora (*Carrier*) que se realiza en escala logarítmica. De esta forma se simula la percepción humana del sonido.

En primer lugar, se sintetizan los N archivos *.wav* mediante la librería **NumPy** y la formulación matemática de la FM (Fase 1). Se define una frecuencia de muestreo (*sample rate*) de 44100 Hz y se incrementa el tiempo de duración de 1 a 2 segundos,

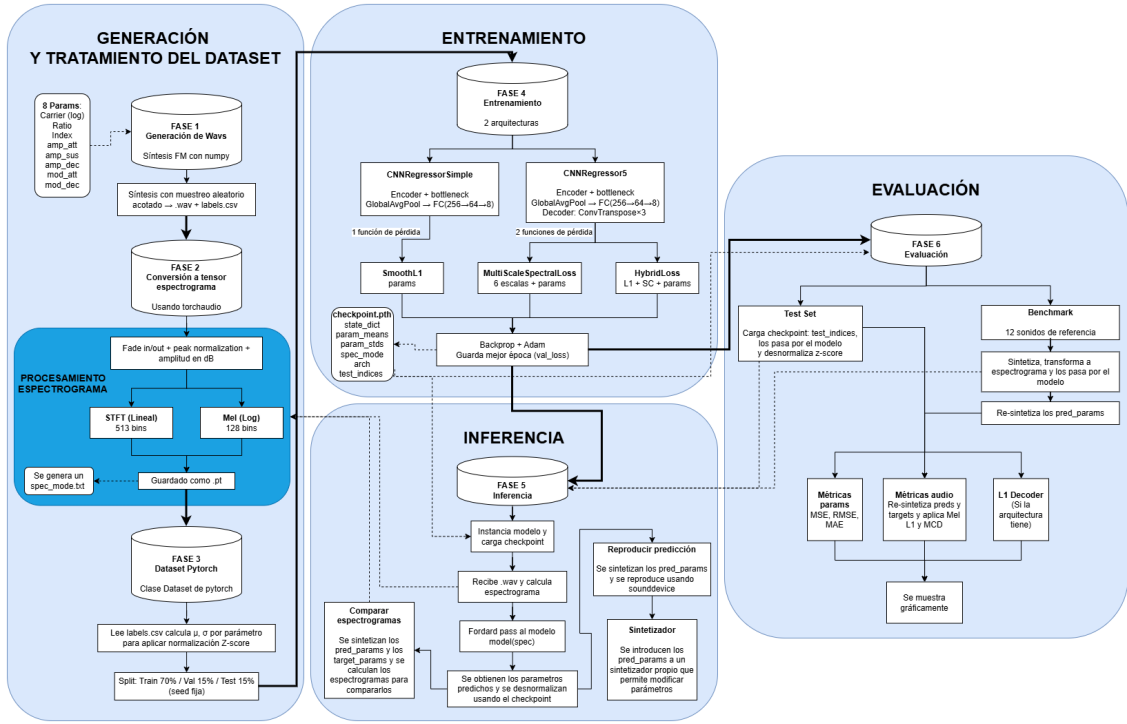


Figura 7.1: Flujo de trabajo del Prototipo 5: desde la síntesis matemática hasta la inferencia y evaluación del modelo.

con el fin de asegurar un margen temporal necesario para analizar y percibir las variaciones en el sonido provocadas por las envolventes. Estas envolventes se generan sobre un vector de tiempo unificado.

En la Fase 2, los audios generados *.wav* se convierten a tensores nativos de *PyTorch*. Se cargan mediante *torchaudio* y se les aplica un suavizado de amplitud en los extremos (*fade-in* y *fade-out*) con el objetivo de evitar chasquidos o ruidos indeseados. Tras una normalización de pico, se aplican las dos transformaciones principales: de audio a espectrograma y de amplitud lineal a logarítmica (dB). Para la representación acústica, el programa permite elegir entre dos métodos, con el fin de llevar a cabo diferentes pruebas de experimentación. Por un lado, se encuentran los espectrogramas en formato lineal, mediante **STFT** con 513 *bins*¹ de frecuencia. Y por el otro lado, se da lugar a elegir un formato logarítmico-perceptual (**Mel** con 128 bandas), el cual tiene un mayor potencial para la representación del sonido de manera similar a la percibida por el humano. Posteriormente, se extrae la magnitud de la señal, se convierte a decibelios (dB) y se guarda como tensores (formato *.pt*). Además, se genera un archivo (*spec_mode.txt*) que indica el tipo de espectrograma utilizado.

Para terminar con el tratamiento del dataset, se estructuran los tensores en una clase Dataset de PyTorch (*SpectrogramTensorDataset*). Se lee el registro de etiquetas (*labels.csv*), el cual contiene todos los valores de los parámetros de cada

¹El término *bin* hace referencia los intervalos de la Transformada de Fourier (FFT). El tamaño de ventana utilizado para la creación de los espectrogramas es de 1024 ($N_{fft} = 1024$), lo que genera $1024/2 + 1 = 513$ intervalos o *bins* de frecuencias

audio, y se calcula la media y desviación estándar global de cada uno de ellos. Con estas estadísticas se aplica una normalización de puntuación estándar (*Z-score*), nivelando así las diferencias entre las escalas de los parámetros, consiguiendo igualar las importancias de cada uno en el cálculo del gradiente. Finalmente, el dataset se divide, mediante una semilla fija, en: 70 % para entrenamiento, 15 % validación y se guarda otro 15 % como conjunto de pruebas (*Test set*).

7.1.2. Entrenamiento

La fase de entrenamiento se caracteriza por permitir la elección de diferentes arquitecturas y funciones de pérdida con el fin de experimentar. De esta forma, se pueden probar diferentes combinaciones para ver cual es más interesante en tiempo o precisión.

- **Arquitecturas:** Se dispone de dos topologías convolucionales diferentes:
 - **CNNRegressorSimple:** Está formada por un *Encoder* de tres bloques convolucionales, un *Bottleneck* y una cabeza de regresión pura (*Global Average Pooling*). Esta última está conectada a un Perceptrón Multicapa que acaba en 8 neuronas. A pesar de que la red ejecuta un entrenamiento rápido, al carecer de reconstrucción no se garantiza que el *Encoder* esté extrayendo de manera acertada la estructura del espectrograma, prediciendo únicamente números.
 - **CNNRegressor5:** Se trata de una expansión del modelo anterior, incorporando una segunda cabeza de predicción. Se añade un *Decoder* que consta de 3 capas convolucionales traspuestas, las cuales tratan de reconstruir el espectrograma a partir del espacio latente. Así pues, esta etapa fuerza al *Encoder* a capturar la riqueza temporal y tímbrica del sonido para evitar un error grande.
- **Funciones de Pérdida:** Para el modelo simple, se evalúa la diferencia paramétrica únicamente mediante **SmoothL1 Loss**. Esta se utiliza por su robustez y versatilidad en la regresión numérica: cuando la diferencia entre el valor real y el predicho es muy pequeña, se comporta como un Error Cuadrático (L2), lo que suaviza la convergencia de gradientes; y ante desviaciones grandes, se asemeja al Error Absoluto (L1), de forma que los casos aislados que se alejan del resto no desestabilicen la red. Esta función de pérdida tiene la desventaja de no tener en cuenta la no-inyectividad de la síntesis FM.

En cuanto al modelo CNNRegressor5, se permite la elección de una de las siguientes funciones de pérdida:

- **HybridLoss:** Es una función de pérdida híbrida, que combina el Error Absoluto Medio (L1) de la reconstrucción del espectrograma, un refuerzo estructural de Convergencia Espectral (SC), y la penalización paramétrica anterior. La primera se lleva el peso principal (de 1.0), calcula el Error Absoluto Medio píxel a píxel entre el espectrograma original y el reconstruido (en decibelios). La Convergencia Espectral (SC) se aplica con un

peso medio (de 0.5) para evitar los espectros borrosos generados por L1. Esta métrica calcula la norma de Frobenius para evaluar la energía global (relación armónica), regularizando el timbre. A la penalización paramétrica se le da una importancia muy baja (0.05), ya que no es demasiado importante debido al carácter no inyectivo de la síntesis FM.

- **MultiScaleSpectralLoss:** Esta función está inspirada en el modelo de síntesis diferencial DDSP. Al estar utilizando espectrogramas, se simulan los distintos tamaños de ventana de Fourier mediante agrupamientos espaciales temporales (*Average Pooling*). Se agrupa en seis factores (de 1, 2, 4, 8, 16 y 32) y se mide el error en cada una de esas escalas, siendo la media aritmética de esos errores el valor de pérdida total. El cálculo del error de cada escala primero se calcula la diferencia sobre las magnitudes lineales, la cual es útil para capturar picos de energía que pueden ser breves. A esto se le suma después la diferencia sobre las magnitudes en decibelios (logarítmico), multiplicada por un factor α de 1.0, por lo que se le está dando la misma relevancia que el error lineal, capturando así perfectamente la envolvente global y los momentos de baja energía. Mediante esta función, se consigue comparar el sonido obtenido del original en varios niveles de resolución simultáneamente. Por último, se le suma el error paramétrico (*SmoothL1*) con un peso muy bajo (0.05) con el objetivo de que la red no deje de tener mínimamente en cuenta los valores de los parámetros.

En la etapa final del entrenamiento, se encuentra el algoritmo de retropropagación (*Backpropagation*), mediante el cual se actualizan los pesos para así minimizar el error en la red, calculando los gradientes del error con respecto a cada nodo. Para la actualización de los pesos se emplea el optimizador **Adam** (*Adaptive Moment Estimation*). Este se elige frente al descenso de gradiente común (SGD) debido al cálculo que hace de las tasas de aprendizaje adaptativas e independientes para cada parámetro. A parte, se guardan los pesos de la época que ha conseguido el mínimo histórico con respecto al error sobre el conjunto de validación. Estos valores se almacenan en un archivo de punto de control (*checkpoint.pth*) junto con el resto de información de contexto necesaria:

- **Parámetros de Desnormalización:** Contiene las medias y desviaciones estándar obtenidas en la Fase 3 sobre los parámetros. Estas son necesarias para poder revertir el Z-score en las futuras predicciones.
- **Contexto de Arquitectura:** Son las etiquetas que describen la topología (*arch*: simple o full) y el tipo de espectrogramas (*spec_mode*: stft o mel) que se han utilizado para el modelo.
- **Aislamiento de Datos:** Contiene los índices del conjunto de prueba (*test_indices*), necesario para la evaluación.

7.1.3. Inferencia

La fase 5 consiste en la inferencia de parámetros de síntesis FM, mediante el modelo entrenado previamente, a partir de audios.

Al inicio de esta fase, se instancia el modelo y carga el punto de control, que instancia automáticamente el tipo de red.

La inferencia comienza cuando se recibe un archivo de audio (*.wav*). En ese momento, se sigue el mismo flujo que en la Fase 2, transformando el audio en un tensor de espectrograma (del tipo indicado por *spec_mode*). Este se introduce en el modelo (*forward pass*), obteniendo así un vector latente de 8 valores, sobre el que se aplica una desnormalización (con los valores de Z-score almacenados en el *checkpoint*) que da como resultado la predicción de los parámetros.

En este punto, el sistema te permite llevar a cabo varias acciones diferentes mediante la interfaz gráfica (GUI). Por un lado, se puede realizar una comparación visual del espectrograma del audio predicho con el del audio original. Para ello, se sintetizan los parámetros obtenidos y los parámetros originales y se calculan sus espectrogramas. También se da la opción de comparar acústicamente el audio original con el sintetizado por los parámetros inferidos (mediante *sounddevice*) y de exportar estos a un sintetizador propio, con la opción de modificarlos manualmente.

7.1.4. Evaluación

La última fase se centra en la evaluación del rendimiento del sistema y en la experimentación en el uso de las diferentes combinaciones de arquitecturas y funciones de pérdida. Para ello, la fase se divide en dos vertientes: evaluación sobre *Test Set* y evaluación sobre *Benchmark*. En la primera, se utilizan las muestras reservadas en la división inicial del dataset (un 15%). Los índices exactos de dicha muestra, se consiguen del archivo de punto de control (*checkpoint*), evitando datos cruzados con los vistos en el entrenamiento. Y en cuanto a la evaluación sobre *Benchmark*, su objetivo es poner a prueba el manejo del modelo dada la característica inyectividad nula de la síntesis FM. Para ello, se diseña un banco de pruebas (*benchmark*) con 12 sonidos de referencia que abarcan todo el espacio tímbrico del sintetizador (desde un bajo limpio a sonidos agudos, complejos y muy ruidosos).

Sobre los resultados obtenidos por las dos vertientes previas, se pueden calcular diferentes métricas para la evaluación del modelo en ambos ámbitos:

- **Métricas Sobre Parámetros:** Estas evalúan el modelo en función de la precisión obtenida en los parámetros inferidos. Se emplean las métricas de: Error Cuadrático Medio (MSE), que penaliza las desviaciones paramétricas grandes entre predicción y original; Raíz del Error Cuadrático Medio (RMSE), que devuelve el error en las unidades originales de cada parámetro, consiguiendo ver de forma directa la precisión y así poder hacer comparaciones objetivas entre las distintas arquitecturas y funciones de pérdida; y Error Absoluto Medio (MAE), el cual es más robusto frente a valores atípicos. Estas métricas sufren ante el problema de la inyectividad nula en la síntesis FM, por lo que se utilizan principalmente como diagnóstico rápido.

- **Métricas de Audio:** Estas son mucho más interesantes para conseguir una evaluación similar a la que daría un oído experto, ya que vuelven a sintetizar los parámetros y realizan una comparativa perceptual de los sonidos. Primero se aplica la Distancia L1 en Espectrogramas Mel (*Mel l1*), la cual calcula el Error Absoluto Medio (L1) sobre las señales previamente transformadas a la escala Mel, consiguiendo así la sensibilidad logarítmica del oído humano. De igual manera se aplica la Distorsión Cepstral Mel (MCD - *Mel-Cepstral Distortion*). Es una métrica que trata de evaluar la similitud del timbre de los audios, extrayendo 13 Coeficientes Cepstrales en las Frecuencias de Mel (MFCC). El primer coeficiente, que representa la energía global, es descartado para que la métrica refleje exclusivamente el timbre (textura del sonido).
- **Métrica de Reconstrucción (Decoder L1):** En el caso en el que se utilice la arquitectura **CNNRegressor5**. Calcula el Error Absoluto Medio (L1) entre el espectrograma original y el reconstruido por la red, de forma que se consigue evaluar la asimilación de información en el espacio latente.

7.2. Interfaz Gráfica de la Aplicación

Para una mejor experiencia de uso y facilitar el entorno de entrenamientos y pruebas de la aplicación, se ha desarrollado una Interfaz Gráfica. Se trata de una serie de pantallas sencillas, programadas en Python mediante el uso de la librería *Tkinter*.

En la primera pantalla (7.2), se da la opción de elegir un modelo existente o de entrenarlo.

En caso de decidir cargar un modelo, se podrá seleccionar mediante el explorador de archivos. A continuación (7.3), existen dos posibilidades: probar predicciones independientes de audios o realizar una evaluación general. En la primera, se puede cargar un wav y generar una predicción, existiendo la posibilidad de compararlos auditivamente o gráficamente sus espectrogramas (en frecuencia lineal y en logarítmica) y de ejecutar un sintetizador. En la pantalla del sintetizador(7.4), aparecen los parámetros de la predicción y los del audio original (en caso de tenerlos), siendo modificables. Además, el teclado hace la función de teclas del sintetizador con el sonido generado por los parámetros seleccionados.

La sección del entrenamiento (7.5) es la dedicada a seleccionar varios aspectos relevantes, tanto para el tiempo y forma de entrenamiento como para la topología del modelo. Por un lado, se permite elegir si entrenar el modelo en CPU o en GPU, lo que afectará directamente a su tiempo de entrenamiento. Por el otro, hay que decidir que tipo de espectrograma, arquitectura del modelo y función de pérdida (explorados en el apartado anterior) van a ser los que definan el modelo. El siguiente campo de la pantalla incluye la opción de crear el dataset (explicado anteriormente) o cargar uno personalizado. En último lugar se encuentra el apartado de parámetros modificables para el entrenamiento, como son las épocas (*epoch*), la tasa de aprendizaje (*Learning Rate*) y los pesos de los parámetros de las funciones de pérdida.

7.3. Estructura del Código

Para asegurar la mantenibilidad y escalabilidad, el código se estructura mediante el principio de Separación de Responsabilidades (*Separation of Concerns*). Se divide así la aplicación en dos módulos diferentes:

- **Lógica (Backend):** Se encuentra en el archivo *logica.py*. Aquí se agrupan las funciones matemáticas para la síntesis, calculo de métricas, generación de espectrogramas, etc.
- **Vista y Controlador (Frontend):** Archivo *main.py*. Este implementa la interfaz, recibe los *inputs* del usuario y gestiona el flujo de llamadas a la lógica y los errores.

Adicionalmente, destaca el uso de la Programación Orientada a Objetos (POO) y el encapsulamiento. Estas características se aplican en la aplicación de la siguiente manera:

- **Clase App (*main.py*):** Se encarga de encapsular el estado global. Evita el uso de variables globales manteniendo las variables necesarias instanciadas en esta clase (*self.dataset_obj*, *self.pathModelo*, *self.model_trained*, etc).
- **Clase SpectrogramTensorDataset (*dataset.py*):** Hereda de la clase base de *Pytorch* de *torch.utils.data.Dataset*, teniendo acceso a métodos como el de procesamiento por lotes (*batching*). Asimismo, esta clase se encarga de los cálculos de estadísticas de normalización y mapeo de los archivos, de forma que desde fuera se obtienen los datos ya preparados mediante el método `__getitem__`.
- **Modelos y Funciones de Pérdida:** Como se ha mencionado previamente, existen diferentes arquitecturas y funciones de pérdida y todas ellas heredan de *nn.Modules*. En consecuencia, el código permite que la lógica del entrenamiento sea independiente del modelo exacto que se esté utilizando, lo que se conoce como **Polimorfismo**. Por otro lado, la clase *HybridLoss* encapsula la lógica de la convergencia espectral y la norma de Frobenius, manteniendo el entrenamiento ajeno a estas fórmulas.
- **FMSynth8Window:** La pantalla del sintetizador se encuentra totalmente desacoplada del resto de la lógica, siendo una clase independiente. De esta manera, se puede utilizar tanto desde la ejecución normal de la aplicación como de forma aislada, facilitando así su construcción, ampliación y pruebas aisladas.

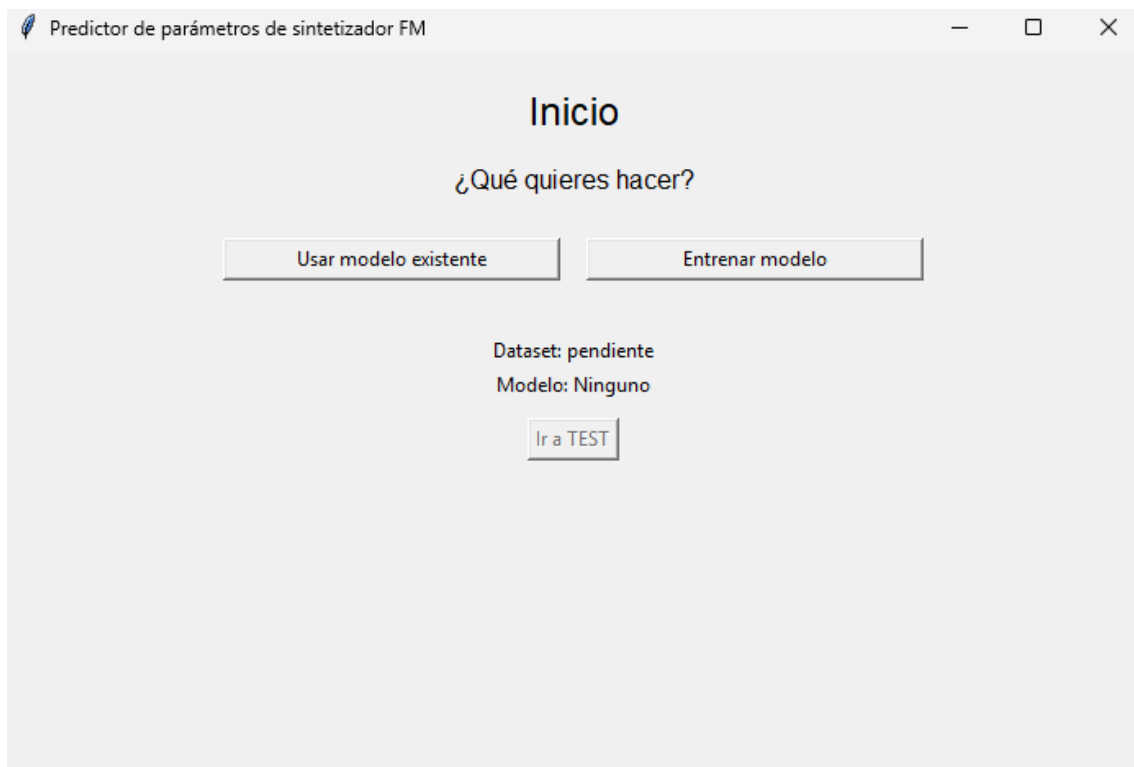


Figura 7.2: Primera pantalla de la Interfaz Gráfica de la Aplicación, elección uso modelo o entrenamiento.

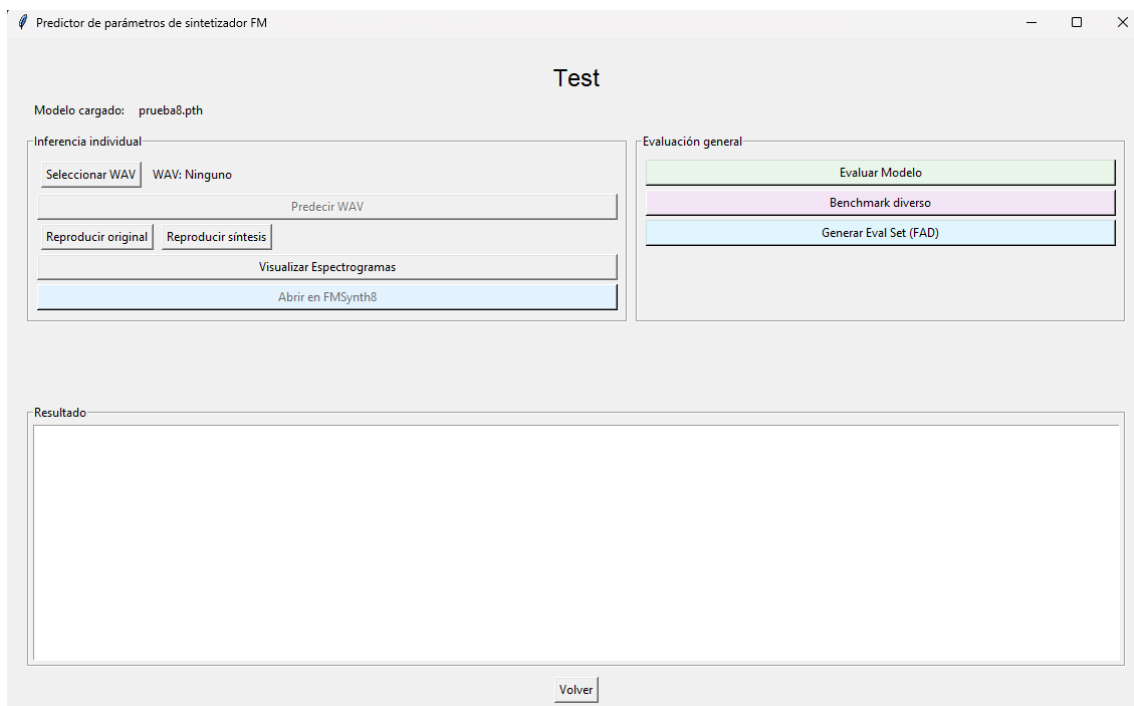


Figura 7.3: Pantalla en la que se pueden hacer pruebas con el modelo seleccionado.

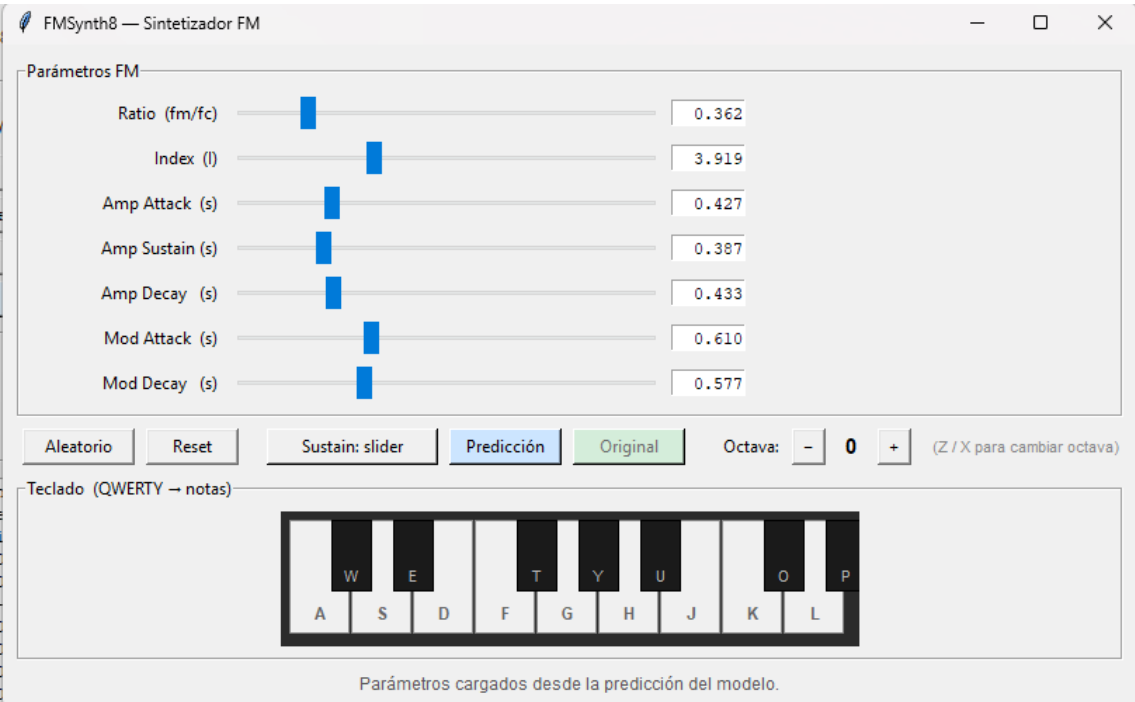


Figura 7.4: Sintetizador FM de prueba.

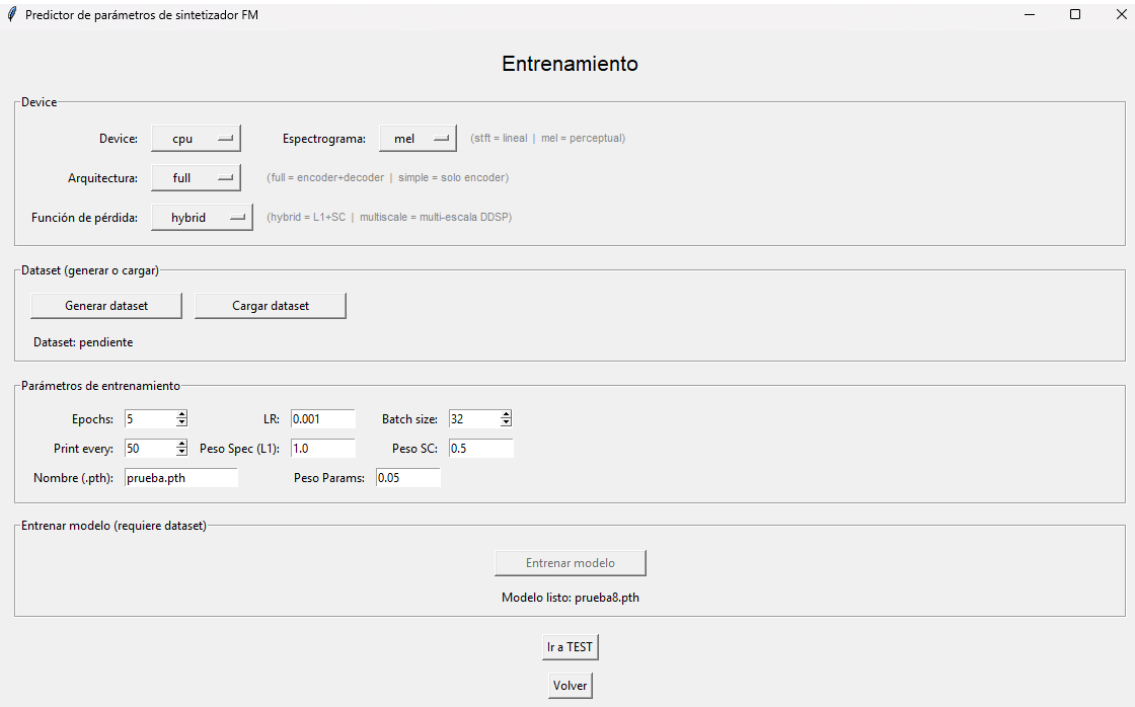


Figura 7.5: Sintetizador FM de prueba.

Resultados y Evaluación

En este capítulo se muestran las pruebas realizadas sobre el modelo. El objetivo es el de evaluación del rendimiento del sistema y en la experimentación en el uso de las diferentes combinaciones de arquitecturas y funciones de pérdida. Para ello, la fase se divide en dos vertientes: evaluación sobre *Test Set* y evaluación sobre *Benchmark*. En la primera, se utilizan las muestras reservadas en la división inicial del dataset (un 15%). Los índices exactos de dicha muestra, se consiguen del archivo de punto de control (*checkpoint*), evitando datos cruzados con los vistos en el entrenamiento. Y en cuanto a la evaluación sobre *Benchmark*, su objetivo es poner a prueba el manejo del modelo dada la característica inyectividad nula de la síntesis FM. Para ello, se diseña un banco de pruebas (*benchmark*) con 12 sonidos de referencia que abarcan todo el espacio tímbrico del sintetizador (desde un bajo limpio a sonidos agudos, complejos y muy ruidosos).

Sobre los resultados obtenidos por las dos vertientes previas, se pueden calcular diferentes métricas para la evaluación del modelo en ambos ámbitos:

- Métricas Sobre Parámetros:** Estas evalúan el modelo en función de la precisión obtenida en los parámetros inferidos. Se emplean las métricas de: Error Cuadrático Medio (MSE), que penaliza las desviaciones paramétricas grandes entre predicción y original; Raíz del Error Cuadrático Medio (RMSE), que devuelve el error en las unidades originales de cada parámetro, consiguiendo ver de forma directa la precisión y así poder hacer comparaciones objetivas entre las distintas arquitecturas y funciones de pérdida; y Error Absoluto Medio (MAE), el cual es más robusto frente a valores atípicos. Estas métricas sufren ante el problema de la inyectividad nula en la síntesis FM, por lo que se utilizan principalmente como diagnóstico rápido.
- Métricas de Audio:** Estas son mucho más interesantes para conseguir una evaluación similar a la que daría un oído experto, ya que vuelven a sintetizar los parámetros y realizan una comparativa perceptual de los sonidos. Primero se aplica la Distancia L1 en Espectrogramas Mel (*Mel l1*), la cual calcula el Error Absoluto Medio (L1) sobre las señales previamente transformadas a la escala Mel, consiguiendo así la sensibilidad logarítmica del oído humano. De igual manera se aplica la Distorsión Cepstral Mel (MCD - *Mel-Cepstral*

Distortion). Es una métrica que trata de evaluar la similitud del timbre de los audios, extrayendo 13 Coeficientes Cepstrales en las Frecuencias de Mel (MFCC). El primer coeficiente, que representa la energía global, es descartado para que la métrica refleje exclusivamente el timbre (textura del sonido).

- **Métrica de Reconstrucción (Decoder L1):** En el caso en el que se utilice la arquitectura **CNNRegressor5**. Calcula el Error Absoluto Medio (L1) entre el espectrograma original y el reconstruido por la red, de forma que se consigue evaluar la asimilación de información en el espacio latente.

8.1. Definición de los Pipelines Experimentales

Como se explicó en el capítulo sobre la arquitectura final (7), el prototipo 5 consta de varias posibles arquitecturas y funciones de pérdida, que se pueden combinar dando lugar a seis diferentes configuraciones (o *Pipelines*). El diagrama 8.1 muestra las diferentes topologías que se dan.

Con la evaluación y comparación de las diferentes configuraciones, se espera determinar cuál de ellas será la "definitiva", la que ofrezca mejores predicciones, teniendo también en cuenta sus tiempos de entrenamiento, la consistencia de sus predicciones y su eficiencia general.

8.2. Características del *Hardware* empleado en el entrenamiento de los modelos

Para la contextualización de los datos de tiempos requeridos en el entrenamiento de los diferentes modelos, se especifican las características de la máquina empleada (cedida por la universidad):

8.3. Evaluación de las Diferentes Arquitecturas y Funciones de Pérdida

La evaluación de las diferentes topologías (7.1.2) se ha llevado a cabo con un único dataset de 50000 audios (método de generación explicado aquí: 7.1.1), un número que ha resultado ser equilibrado, permitiendo unos resultados decentes sin suponer un incremento demasiado grande en los tiempos de entrenamiento de los modelos.

El número de épocas no se ha mantenido fijo, se ha utilizado un valor de paciencia (*Early Stopping*) de 10 épocas. De esta manera, se comprueba el error de validación en cada época y si después de 10 seguidas no se ha mejorado su record histórico, la red deja de entrenarse. Así pues, se evita la Sobreadaptación (*Overfitting*) de la red neuronal, puesto que se evita un entrenamiento excesivo, y se ahorra tiempo.

Por otro lado, se ha elegido un *Batch Size* de tamaño 32. Este, aparte de ser un estándar, es un valor asumible por cualquier tarjeta gráfica moderna y que consigue

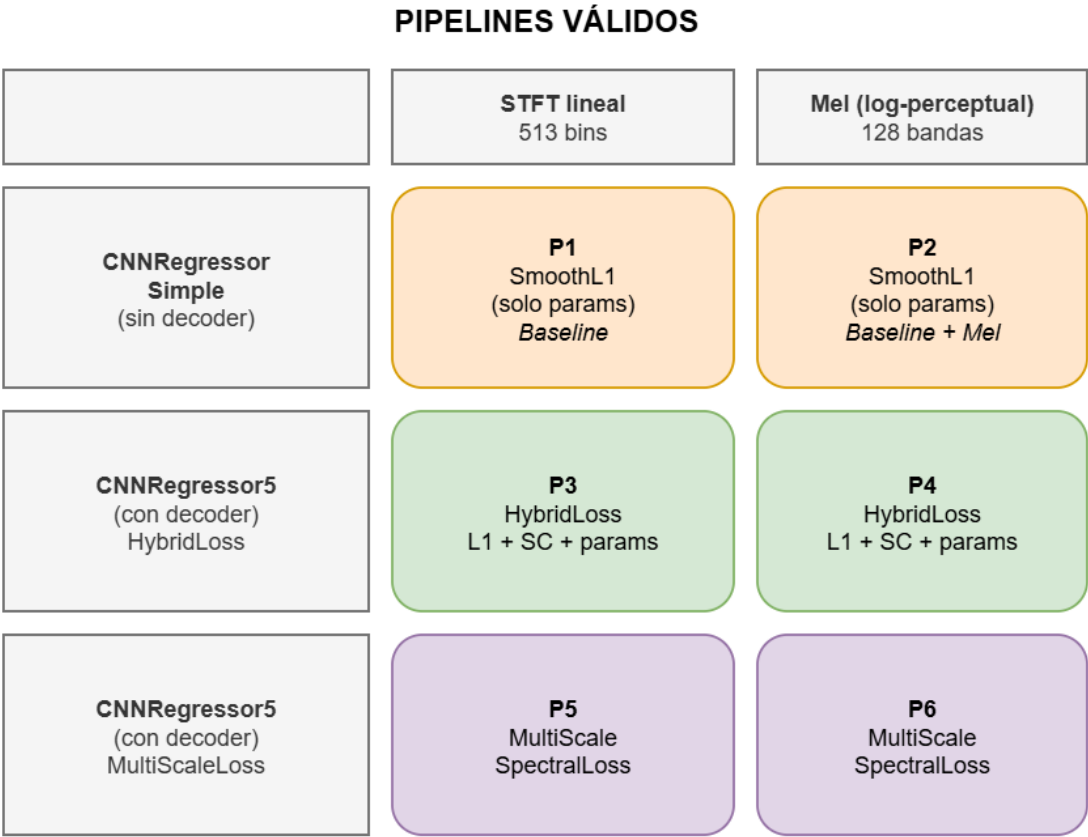


Figura 8.1: Representación de las diferentes combinaciones de Arquitecturas y Funciones de Pérdida.

una eficiencia adecuada.

Benchmark:

- **Tamaño del Dataset:** 50000
- **Valor de Paciencia:** 10
- **Batch Size:** 32

8.3.1. P1: CNNRegressorSimple: SmoothL1, STFT lineal

8.3.1.1. Evaluación Mediante *Test Set*

8.3.1.2. Evaluación Mediante *Benchmark* Diverso

8.3.2. P2: CNNRegressorSimple: SmoothL1, Mel (log-perceptual)

El modelo con estructura de *CNNRegressorSimple*, con función de pérdida *SmoothL1* (explicada también en el apartado 7.1.2) y espectrogramas Mel ha necesitado 57 minutos y 57 épocas para entrenarse, acabando con un error de validación 0.006844. Este valor indica que la red está consiguiendo predecir los parámetros con una precisión altísima. Sin embargo, debemos de tener en cuenta que en la síntesis FM, una pequeña desviación en un parámetro como índice de modulación, podría ocasionar la producción de un sonido completamente diferente al original. Además, cabe destacar que mediante la función de pérdida *SmoothL1*, se está ignorando la inyectividad nula que caracteriza a FM.

8.3.2.1. Evaluación Mediante *Test Set*

En la gráfica 8.2, se muestran los resultados de la evaluación del modelo con las métricas Mel L1 y MCD, sobre el conjunto de test (*Test Set*) del dataset.

8.3.2.2. Evaluación Mediante *Benchmark* Diverso

Al aplicar *Mel L1* y *MCD* sobre los resultados del *benchmark*, obtenemos el gráfico 8.3. A simple vista se aprecia que hay dos casos en los que las métricas, previamente mencionadas, se encuentran muy por debajo de la media. Estos casos son con el sonido *bajo_limpio* y con *muy_bajo*. Este hecho se debe a que las frecuencias bajas y limpias poseen patrones visuales muy claros y fáciles de asimilar para la red. Por ejemplo, si se observa el espectrograma de *muy_bajo* y el de la predicción de la red ante el mismo 8.4, se puede apreciar la claridad y similitud entre el original y la predicción.

En contraposición, se hayan los resultados sobre el audio *pad_lento*, que destaca por haber generado un incremento de ambas métricas. Un *pad* es un sonido de sintetizador caracterizado por comenzar muy lento (ataque bajo) y mantenerse. Comparando el espectrograma original de *pad_lento* y el de la predicción podemos sacar algunas conclusiones de estos resultados. Como se ve en la figura 8.5, la red ha conseguido inferir perfectamente los parámetros relacionados con la envolvente,

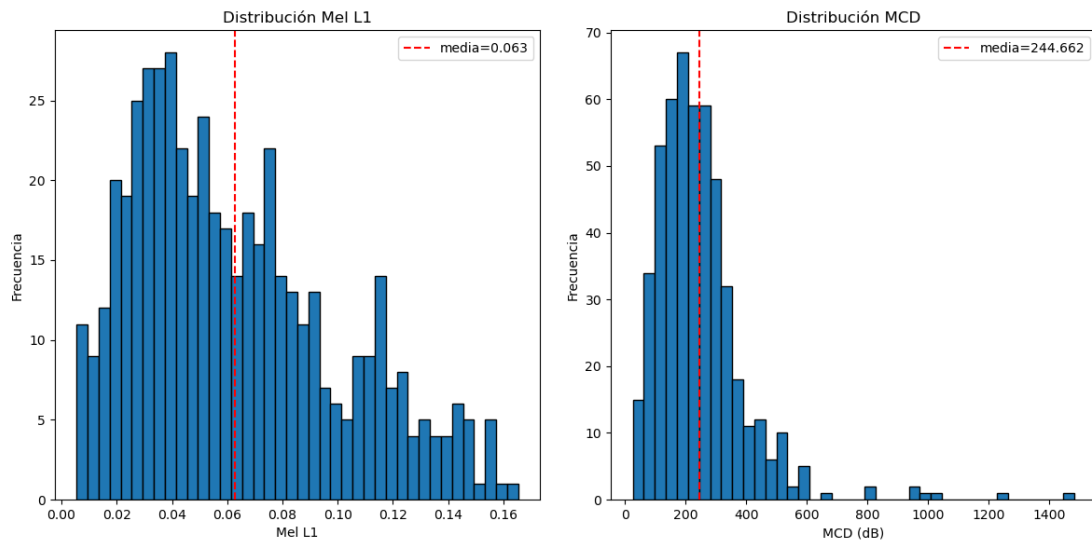


Figura 8.2: Graficas que muestran los resultados de evaluar el conjunto de tests sobre el modelo P2 con las métricas Mel L1 y MCD.



Figura 8.3: Gráfico obtenido en la evaluación de P2 mediante *benchmark*.

Comparativa de Espectrogramas: Original vs Predicción [Mel (log freq)]

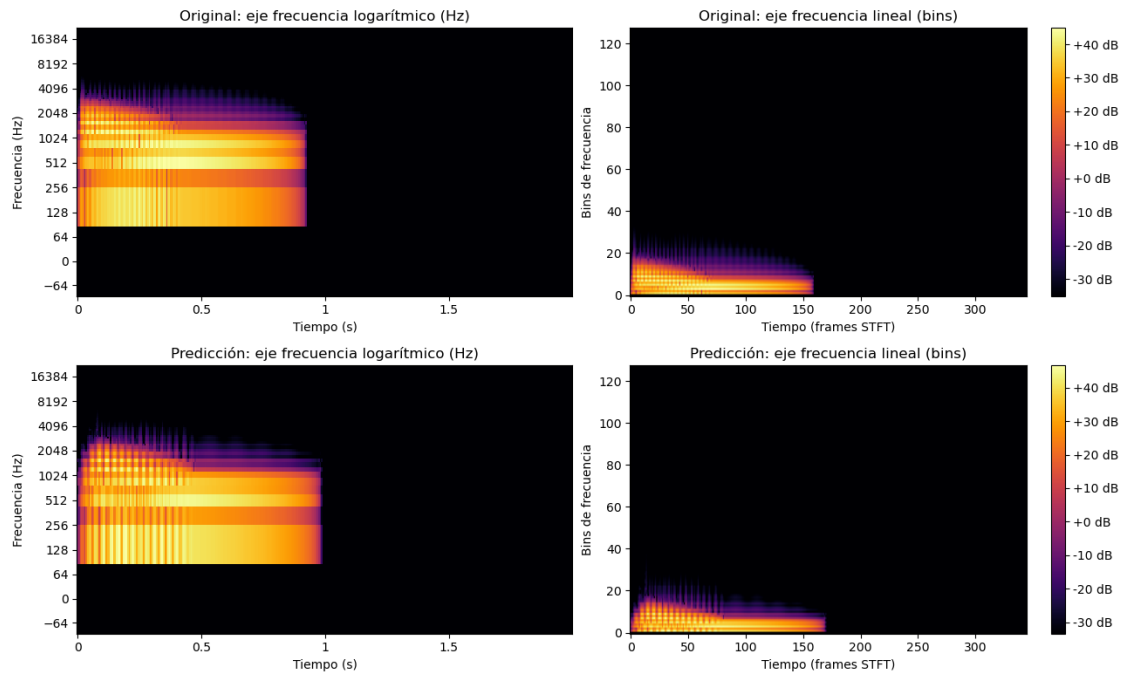


Figura 8.4: Espectrograma resultado de la predicción del audio de Benchmark: *muy_bajo*.

Comparativa de Espectrogramas: Original vs Predicción [Mel (log freq)]

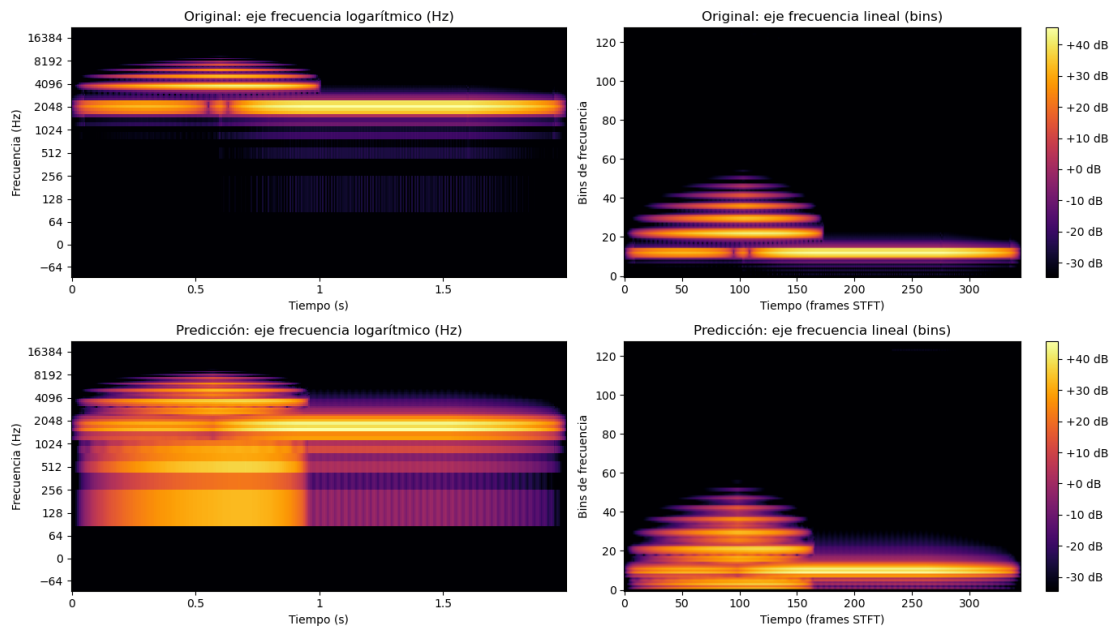


Figura 8.5: Espectrograma resultado de la predicción del audio de Benchmark: *muy_bajo*.

pero ha fallado considerablemente en la búsqueda de las frecuencias y los parámetros que las afectan, de los que depende el timbre (frecuencia). Por otra parte, también se puede llegar a la conclusión lógica de que, al ser un sonido de gran duración, se puede acumular un mayor error en las métricas. Esto es debido a que, a pesar de que dichas métricas calculan la media, como a la red identifica con gran precisión los silencios, los audios que contengan una mayor parte en silencio van a conseguir mejores métricas.

Media Obtenida en las Métricas:

- **Mel L1 por sonido:** 0.028

- **MCD por sonido:** 214.520

8.3.3. P3: CNNRegressor5: HybridLoss, STFT lineal

8.3.3.1. Evaluación Mediante *Test Set*

8.3.3.2. Evaluación Mediante *Benchmark* Diverso

8.3.4. P4: CNNRegressor5: HybridLoss, Mel (log-perceptual)

La red neuronal con estructura *CNNRegressor5*, función de pérdida híbrida (*HybridLoss*) y espectrogramas Mel, ha requerido de un total de 27 épocas para entrenarse, consiguiendo reducir el error de validación a 0.672. Este valor de error es indicativo de un entrenamiento exitoso. Como la función de pérdida híbrida da prioridad a la diferencia media (L1) entre el espectrograma original y el reconstruido (con un peso de 1.0), significa que se está calculando una diferencia media de menos de 0.672 decibelios por pixel, confirmando una gran similitud entre los espectrogramas. Al tratarse de espectrogramas de Mel, que consiguen una representación similar a la forma de percibir los audios de los humanos, se puede esperar que sean muy similares acústicamente.

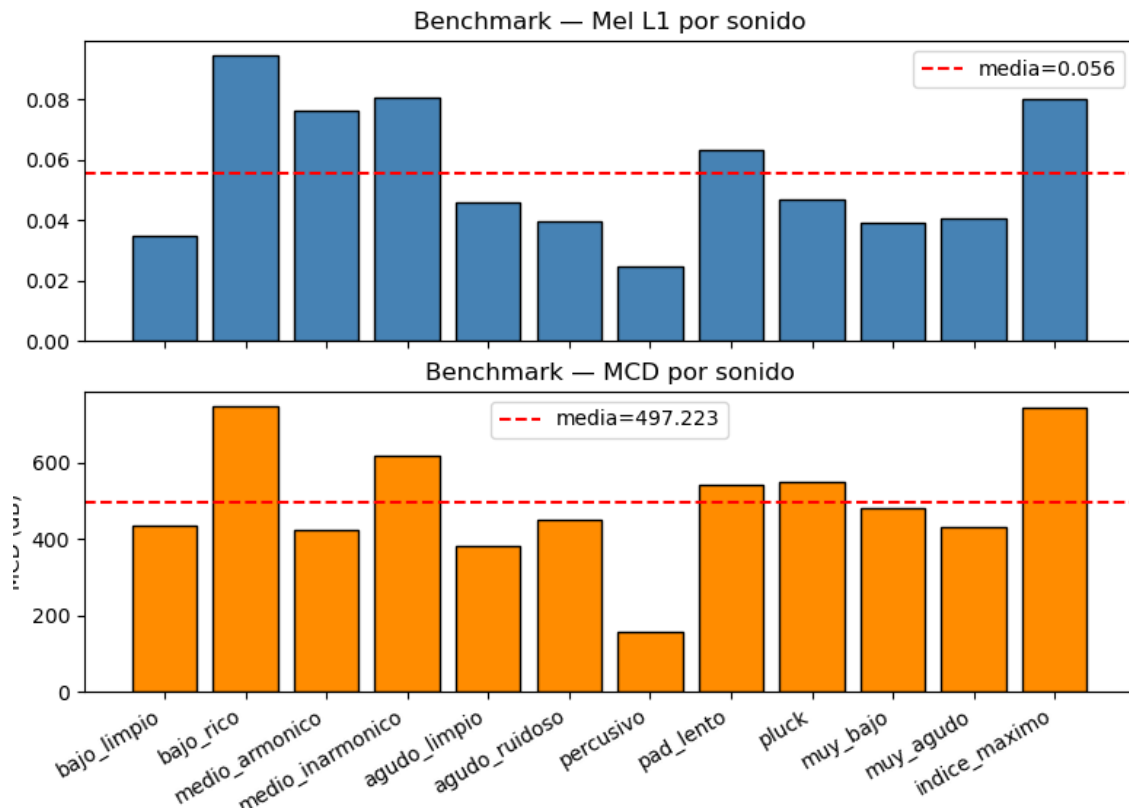


Figura 8.6: Gráfico obtenido en la evaluación de P3 mediante *benchmark*.

8.3.4.1. Evaluación Mediante *Test Set*

8.3.4.2. Evaluación Mediante *Benchmark* Diverso

8.3.5. P5: CNNRegressor5: MultiScaleSpectralLoss, STFT lineal

8.3.5.1. Evaluación Mediante *Test Set*

8.3.5.2. Evaluación Mediante *Benchmark* Diverso

8.3.6. P6: CNNRegressor5: MultiScaleSpectralLoss, Mel (log-perceptual)

8.3.6.1. Evaluación Mediante *Test Set*

8.3.6.2. Evaluación Mediante *Benchmark* Diverso

8.4. Comparación de los Resultados Obtenidos

Pipeline	Métricas Paramétricas (RMSE) ↓			Métricas Acústicas ↓		
	Carrier (Hz)	Ratio	Index	Mel L1 (dB)	MCD	Decoder L1
P1	X.XX	X.XX	X.XX	X.XX	X.XX	N/A
P2	X.XX	X.XX	X.XX	X.XX	X.XX	N/A
P3	X.XX	X.XX	X.XX	X.XX	X.XX	X.XX
P4	X.XX	X.XX	X.XX	X.XX	X.XX	X.XX
P5	X.XX	X.XX	X.XX	X.XX	X.XX	X.XX
P6	X.XX	X.XX	X.XX	X.XX	X.XX	X.XX

Tabla 8.1: Comparativa de métricas paramétricas y acústicas sobre el Test Set para los 6 pipelines. Las flechas (↓) indican que un valor menor representa un mejor rendimiento. El Decoder L1 no aplica (N/A) en las arquitecturas simples.

Conclusiones y Trabajo Futuro

A través de este proyecto se ha demostrado la viabilidad de utilizar **Redes Neuronales Convolucionales (CNNs)** para automatizar la estimación de parámetros en la **síntesis FM**. El sistema desarrollado logra reducir la barrera técnica del *sound matching*, confirmando que el **aprendizaje profundo** es una herramienta eficaz para agilizar el flujo de trabajo de productores y diseñadores de sonido, permitiéndoles centrarse en el proceso creativo.

Un trabajo de esta naturaleza es fácilmente continuable desde múltiples perspectivas; sin embargo, consideramos relevante destacar las cuatro ramas de investigación y desarrollo más prometedoras:

1. **Implementación de un sintetizador diferenciable:** La integración de librerías como DDSP (*Differentiable Digital Signal Processing*) permitiría integrar el proceso de síntesis directamente en el entrenamiento de la red. Esto posibilitaría un cálculo exacto en el descenso de gradiente (al no tratar el sintetizador como una caja negra), optimizando la convergencia del modelo y mejorando notablemente la fidelidad de los resultados.
2. **Escalabilidad a otros métodos de síntesis:** El flujo de trabajo establecido sienta las bases para expandir el sistema más allá de síntesis por modulación de frecuencia. Sería de gran interés adaptar el modelo para estimar parámetros en síntesis aditiva, sustractiva, de modelado físico o granular.
3. **Exploración arquitectónica y optimización:** Para superar los límites de rendimiento actuales, se propone un estudio más exhaustivo de hiperparámetros, la evaluación de arquitecturas de red alternativas y el diseño de funciones de pérdida personalizadas. Esto permitiría aportar enfoques propios y novedosos, reduciendo la dependencia de los modelos estandarizados en la literatura actual.
4. **Desarrollo de un plugin VST:** De cara a su viabilidad comercial y práctica, el paso lógico es encapsular el modelo en un formato VST para su integración directa en cualquier DAW (*Digital Audio Workstation*). Esto requerirá el diseño de una interfaz gráfica de usuario (GUI) intuitiva y la programación de un sistema más robusto para el manejo de excepciones, evitando que la aplicación

falle ante entradas imprevistas o casos límite (*edge cases*) por parte del usuario final.

5. **Evaluación perceptual mediante pruebas de escucha:** Aunque el rendimiento del modelo se ha medido utilizando métricas cuantitativas, la incorporación de una validación cualitativa a través de usuarios reales supondría un avance significativo. La realización de pruebas de percepción subjetiva con productores y diseñadores de sonido permitiría evaluar la similitud tímbrica desde una perspectiva psicoacústica real, aportando una métrica de éxito empírica y directamente alineada con el oído humano.

Chapter 10

Introduction

“I dream of instruments obedient to my thought and which with their contribution of a whole new world of unsuspected sounds, will lend themselves to the exigencies of my inner rhythm.”

— Edgard Varèse

10.1. Motivation

Modern music production has undergone a radical transformation in recent decades, consolidating the computer and software as the central axes of the creative process. In this digital paradigm, the generation of artificial audio signals through synthesizers has become an indispensable tool for artists, producers, and sound designers. Historically, early analog synthesizers, based on additive¹ and subtractive² synthesis methods, offered a relatively accessible workflow; their physical and visual interface, controlled through potentiometers and filters, allowed musicians to sculpt sound intuitively and directly.

However, the synthesis landscape changed drastically in the late 1960s with the discovery of Frequency Modulation (FM) synthesis and, on a massive scale, with the commercialization of the iconic **Yamaha DX7** in 1983. This technology represented a revolution in the market by enabling the creation of metallic, electric, and bell-like timbres, acoustically impossible to achieve with the technology of the time. Nevertheless, this innovation brought with it a significant dilemma: **its programming is notoriously counterintuitive**. Unlike traditional systems, in FM synthesis a minimal change in a single parameter (such as the modulation index or the frequency of an operator) can completely destroy the harmonic structure of the sound. This extreme parametric sensitivity and steep learning curve have historically alienated most creators from FM sound design, relegating them to the use of preconfigured *presets*.

¹Additive synthesis seeks to construct sounds by combining (adding, i.e., ‘by addition’) simpler elements than the original sound. (Hispasónica, 2026).

²It is a synthesis method where a signal is generated by an oscillator and then filtered (Wikipedia, 2026).

This level of technical complexity manifests its greatest obstacle in the process known as ***Sound Matching*** or timbral matching. It is an everyday situation in the recording studio for a producer to hear a specific sound and wish to replicate it. Performing this task “by ear” with an FM synthesizer is a trial-and-error process that can be deeply frustrating. From a mathematical and acoustic perspective, this problem lies in the **lack of injectivity** of the synthesis engine: drastically different parameter combinations can generate acoustic results that the human ear perceives as identical, while minuscule adjustments can result in completely opposite sonic textures. Consequently, manual sound design becomes a technical barrier that interrupts the artist’s creative flow.

Faced with this problem, Artificial Intelligence and, specifically, **Deep Learning**, present themselves as the ideal bridge between the musician’s creative abstraction and the mathematical complexity of the synthesizer. By delegating parameter inference to a computational model, the search for the target sound can be automated. To achieve this, the system must be able to “listen” to and process the signal in a way that the network can assimilate. By transforming the one-dimensional audio into a **spectrogram**, the sound is “photographed,” adapting the frequencies to a logarithmic scale that faithfully mimics the perception of the human auditory system.

Once the audio is represented as a two-dimensional image, **Convolutional Neural Networks (CNNs)** emerge as the ideal architecture to tackle the problem. These networks are designed to extract spatial features and, in the spectrogram domain, are exceptionally accurate at “seeing” and classifying the dense timbral and harmonic textures of the audio.

10.2. Objectives

General Objective:

To develop a system based on **Convolutional Neural Networks (CNNs)** capable of automatically estimating the parameters of a **frequency modulation (FM)** synthesizer from an audio sample, in order to replicate its timbre (*sound matching*). Furthermore, the use of different spectrogram representations, network architectures, and loss functions will be studied and compared.

Specific Objectives:

- **Understanding FM synthesis:** Study and understand the parametric architecture of FM synthesis, from its simplest version to a more complex one involving amplitude and modulation envelopes.
- **Literature review:** Research and read the literature related to the use of **Artificial Intelligence applied to audio** to make the fundamental design decisions for the project.
- **Dataset generation:** Design and implement a synthesis engine capable of generating a massive *dataset* of synthetic audio (**.wav**) and their corresponding labels (parameters), as well as their conversion to PyTorch tensors (**.pt**).

- **Signal processing:** Establish a signal processing *pipeline* to transform the raw audio into visual representations (**spectrograms**) suitable for the CNN, applying the data transformations and normalizations that maximize its performance.
- **Model development:** Design, train, and optimize various Convolutional Neural Network architectures and loss functions in Python using **PyTorch**.
- **Psychoacoustic evaluation:** Evaluate the model's performance through parameter error metrics and **timbral similarity** measures that simulate human ear perception.
- **Interface development:** Develop a system with a functional interface that allows accessing all the project's utilities easily and intuitively.

10.3. Work Plan

The development of this Bachelor's Thesis spans from September 2025 to May 2026. The evolution of the project throughout these months is divided into five clearly defined stages:

- **Phase 1 (September - October): Exploration and proof of concept**
Main milestone: General research stage on Artificial Intelligence applied to music.
 During these months, we began laying the technical foundations of deep learning and its application to audio by reading current literature. We started doing very basic tests to see what AI was capable of. After considering options such as a smart mixing assistant or a sound effect generator, we finally focused on its application to synthesizers. The first two prototypes reflect our progress in this new field for us, experimenting with waveform prediction and spectrogram generation.
- **Phase 2 (November): Project definition, *Sound Matching*, and first functional prototypes**
Main milestone: Decision for FM *Sound Matching* and achievement of the first real prototype.
 Here the project took its final direction. We opted for Frequency Modulation (FM) Synthesis and began generating massive labeled *datasets*. We trained a first CNN regression model whose initial results were poor, leading us to restructure the code, refactor it into Object-Oriented Programming (OOP), and create a basic Graphical User Interface (GUI). Finally, we achieved the first functional *sound matching* test, although still with issues in low frequencies and a lack of consistency.
- **Phase 3 (December - February): Complexity scaling and evaluation metrics**
Main milestone: Synthesis improvement and model evaluation.

Once the base was working, we added complexity by incorporating new envelope and logarithm parameters into the sound generation. We reimplemented the dataset generation for this new paradigm (from 3 to 8 parameters). In addition, we unified data processing throughout the program to ensure consistency in obtaining spectrograms and the normalization necessary for the neural network's correct operation. We also implemented the first validation metrics, the FAD (*Fréchet Audio Distance*) metric to evaluate the results, and tools to visualize and compare the spectrograms.

- **Phase 4 (February - April): Architectural optimization and beginning of the thesis**

Main milestone: Deep changes in the neural network and start of writing. These were the months where the most work was put into developing the final prototype. In this phase, we began compiling everything noted and writing the thesis document. At the code level, we made a leap in quality: we implemented Mel scale spectrograms, tested an architecture without a decoder, created a new loss function based on windowing, and programmed an FM benchmark. At the documentation level, we wrote up the progress of prototypes 1 through 3 and began writing the theory on the Mel scale and MFCC coefficients.

- **Phase 5 (April - May): Final drafting, training of definitive models, and closure**

Main milestone: Completion of the thesis and training of the production models (P1 to P6).

The main code development finished; everything programmed in these months was to cover thesis needs (evaluation metrics, early stopping for the network, and training history logging). We fully focused on writing the document, creating diagrams, cleaning up the code, compiling the bibliography, and preparing the scripts for Linux and Windows. To conclude, we trained the six final models using the machines provided by the UCM.

Conclusions and Future Work

Through this project, the feasibility of using **Convolutional Neural Networks (CNNs)** to automate parameter estimation in **FM synthesis** has been demonstrated. The developed system succeeds in reducing the technical barrier of *sound matching*, confirming that **deep learning** is an effective tool to streamline the workflow of producers and sound designers, allowing them to focus on the creative process.

A work of this nature can be easily continued from multiple perspectives; however, we consider it relevant to highlight the five most promising branches of research and development:

1. **Implementation of a differentiable synthesizer:** The integration of libraries such as DDSP (*Differentiable Digital Signal Processing*) would allow the synthesis process to be integrated directly into the network training. This would enable an exact calculation in gradient descent (by not treating the synthesizer as a "black box"), optimizing model convergence and significantly improving the fidelity of the results.
2. **Scalability to other synthesis methods:** The established workflow lays the groundwork for expanding the system beyond frequency modulation synthesis. It would be of great interest to adapt the model to estimate parameters in additive, subtractive, physical modeling, or granular synthesis.
3. **Architectural exploration and optimization:** To overcome current performance limits, a more exhaustive study of hyperparameters, the evaluation of alternative network architectures, and the design of custom loss functions are proposed. This would allow for the contribution of novel, proprietary approaches, reducing the reliance on standardized models in current literature.
4. **Development of a VST plugin:** Looking toward its commercial and practical viability, the logical step is to encapsulate the model in a VST format for direct integration into any DAW (*Digital Audio Workstation*). This will require the design of an intuitive graphical user interface (GUI) and the programming of a more robust system for exception handling, preventing the application from crashing due to unforeseen inputs or *edge cases* from the end user.

5. **Perceptual evaluation through listening tests:** Although the model's performance has been measured using quantitative metrics, incorporating qualitative validation through real users would represent a significant advancement. Conducting subjective perception tests with producers and sound designers would allow for the evaluation of timbral similarity from a real psychoacoustic perspective, providing an empirical success metric directly aligned with human hearing.

Contribuciones Personales

David Cendejas Rodríguez

Mis principales contribuciones a este trabajo se han centrado en el liderazgo del desarrollo técnico, la investigación de arquitecturas de Deep learning y la implementación de los diversos prototipos que han dado forma al sistema final, el prototipo 5. Mi labor ha abarcado desde el planteamiento arquitectónico inicial hasta el entrenamiento y validación de los modelos, asumiendo la responsabilidad de investigar las tecnologías de Machine Learning y Deep Learning aplicadas al ámbito del audio. Este proceso ha supuesto un reto constante, especialmente al abordar la síntesis FM, debido a problemas intrínsecos como la nula inyectividad y la extrema sensibilidad de sus parámetros, factores que han condicionado directamente la convergencia y el rendimiento de nuestro modelo de Sound Matching.

Debido a ello, también he sido el principal responsable del desarrollo del contenido del Resumen, Introducción, Estado de la Cuestión y Conclusiones y Trabajo Futuro de la memoria, además he realizado los diagramas necesarios para las partes más técnicas.

Paralelamente, he llevado a cabo una exploración técnica profunda sobre las representaciones de los datos y las estrategias de normalización. Esta parte del trabajo ha sido fundamental para transformar señales de audio en estructuras de datos procesables, como tensores espectrograma, asegurando la estabilidad del entrenamiento. En este sentido, diseñé e implementé pipelines robustos para el preprocesamiento de audio, garantizando que la entrada al modelo fuera óptima para el aprendizaje de las características tímbricas y unificarlo a los procesos de inferencia y evaluación, permitiendo que el sistema se comportara y escalara de forma coherente en todas sus fases del desarrollo.

En lo relativo a la ingeniería de software, he sido el principal responsable de la implementación del código en Python, aplicando principios de Programación Orientada a Objetos para garantizar un sistema modular y escalable. Debido a la naturaleza investigadora y no lineal del proyecto, el código evolucionó de forma orgánica tras cada reunión con los tutores y la revisión de literatura técnica. Esto me exigió realizar constantes tareas de abstracción y refactorización para evitar la redundancia y mantener la encapsulación, asegurando siempre la eficiencia computacional y la legibilidad del software. Asimismo, gestioné íntegramente las fases de depuración y

manejo de errores, lo que resultó crucial para asegurar la integridad de los resultados obtenidos en un entorno de desarrollo tan dinámico.

Finalmente, al disponer del hardware con mayor capacidad de cómputo del equipo, y a su vez el miembro que se conectó vía ssh a las máquinas proporcionadas por la universidad, asumí la responsabilidad técnica del entrenamiento de los modelos usados por mi compañero para realizar las pruebas y comparaciones.

Ángel Jiménez Izquierdo

Al menos dos páginas con las contribuciones del estudiante 2. En caso de que haya más estudiantes, copia y pega una de estas secciones.

Bibliografía

- ALGFISICA. 10. teorema de fourier. Disponible en https://lalgfisica.readthedocs.io/es/latest/Oscilaciones_Termo/40_Fourier.html.
- ALLEN, J. B. Short term spectral analysis, synthesis, and modification by discrete fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, Disponible en <https://ieeexplore.ieee.org/document/1162950>.
- BARKAN, O., TSIRIS, D. y KOENIGSTEIN, N. Inversynth: Deep neural network for musical synthesizer parameter extraction. En *IEEE/ACM Transactions on Audio, Speech, and Language Processing*. 2019.
- CHOWNING, J. M. The synthesis of complex audio spectra by means of frequency modulation. *Journal of the Audio Engineering Society*, 1973.
- DAVIS, S. B. y MERMELSTEIN, P. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 1980.
- ENGEL, J., HANTRAKUL, L., GU, C. y ROBERTS, A. DDSP: Differentiable digital signal processing. En *International Conference on Learning Representations (ICLR)*. 2020.
- ENGEL, J., RESNICK, C., ROBERTS, A., SANDER, D., NOROUZI, M. y ECK, D. Neural audio synthesis of musical notes with wavenet autoencoders. En *Proceedings of the 34th International Conference on Machine Learning (ICML)*. PMLR, 2017.
- GONG, Y., CHUNG, Y.-A. y GLASS, J. AST: Audio spectrogram transformer. En *Proc. Interspeech 2021*. 2021.
- GREEN MUSICIANS. Dexed. Disponible en <https://www.greenmusicians.com/es/vst-gratuito-de-sintetizador/asb2m10/4459-asb2m10-dexed-3701701228270.html>.
- HISPASONIC. Yamaha dx7. Disponible en <https://www.hispasonic.com/productos/yamaha-dx7/19868>.

- HISPASÓNIC. Síntesis (5): síntesis aditiva. Disponible en <https://www.hispasonic.com/tutoriales/sintesis-5-sintesis-aditiva/38297>.
- KONG, Q., CAO, Y., IQBAL, T., WANG, Y., WANG, W. y PLUMBLEY, M. D. Panns: Large-scale pretrained audio neural networks for audio pattern recognition. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2020.
- KRIZHEVSKY, A., SUTSKEVER, I. y HINTON, G. E. Imagenet classification with deep convolutional neural networks. En *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2012.
- LECUN, Y., BOTTOU, L., BENGIO, Y. y HAFFNER, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 1998.
- VAN DEN OORD, A., DIELEMAN, S., ZEN, H., SIMONYAN, K., VINYALS, O., GRAVES, A., KALCHBRENNER, N., SENIOR, A. y KAVUKCUOGLU, K. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.
- STEVENS, S. S., VOLKMANN, J. y NEWMAN, E. B. A scale for the measurement of the psychological magnitude pitch. *The Journal of the Acoustical Society of America*, Disponible en <https://doi.org/10.1121/1.1915893>.
- VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A. N., KAISER, Ł. y POLOSUKHIN, I. Attention is all you need. En *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2017.
- WIKIPEDIA. Audio bit depth. Disponible en https://en.wikipedia.org/wiki/Audio_bit_depth.
- WIKIPEDIA. Espectrograma. Disponible en <https://es.wikipedia.org/wiki/Espectrograma>.
- WIKIPEDIA. Síntesis sustractiva. Disponible en https://es.wikipedia.org/wiki/S%C3%ADntesis_substractiva.
- YEE-KING, M., FEDDEN, L. y DÍVERNO, M. Automatic programming of VST sound synthesizers using deep networks and other techniques. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2018.